

ECE8803-AI4

Module: Defect Classification

Lecture 1: Wafer Maps — The Obvious Approach and What It Hides

Course website: <https://aikhan123.github.io/khan-lab-website/ai-semiconductors-course.html>

YouTube playlist: <https://www.youtube.com/watch?v=JOOhPoMm58I&list=PLF-ca21MPQt4>

Asif Khan

Associate Professor

School of Electrical and Computer Engineering

School of Materials Science and Engineering

Georgia Institute of Technology

akhan40@gatech.edu

One idea, repeated four times

*Every model encodes an assumption about the physics.
Its failures tell you which assumption broke.*

- We will build four models on the same data — logistic regression, a CNN, hand-engineered polar features, and advanced methods.
- Each one succeeds and fails in a specific, physically interpretable place.
- **If you leave with only this sentence, these two lectures worked — it generalizes to the rest of the course.**

ROADMAP

Where these two lectures go

1

Logistic regression

ASSUMES

pixels are independent

BREAKS ON

position varies → fails

2

Deep neural net

ASSUMES

depth fixes the boundary

BREAKS ON

position still varies

3

CNN

ASSUMES

locality + translation
invariance

BREAKS ON

precision on ambiguous
classes

4

Polar features

ASSUMES

patterns are radially symmetric

BREAKS ON

a line has no radial signature

5

Advanced methods

ASSUMES

each relaxes a prior
assumption

BREAKS ON

the frontier

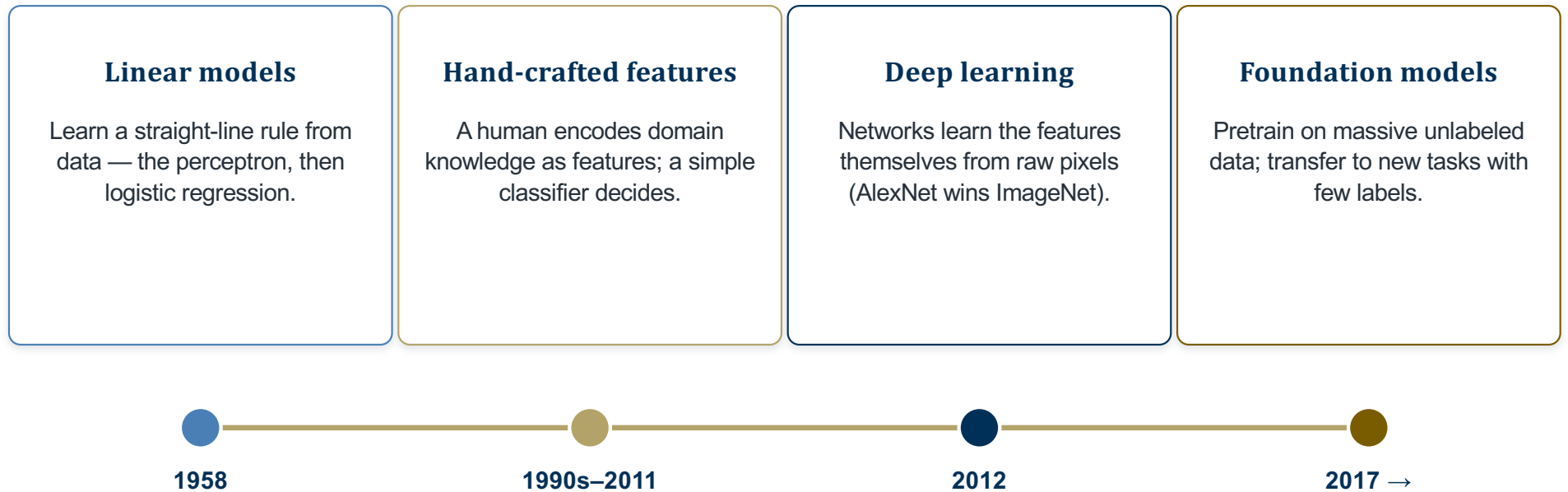
Takeaway Part 1 covers steps 1–3. Part 2 opens with a reversal on step 4.

INTERLUDE · THE CONTEXT

How AI learned to see

Four eras of machine perception — and why today's four models retrace them.

Four eras on one timeline



Takeaway Rules → features → learned features → learned priors: the same path these two lectures walk on wafer maps.

THE “SHALLOW” PARADIGM

Eras 1–2: rules, then hand-crafted features

Linear models (1958 →)

The perceptron learned to separate two classes with a straight line; logistic regression is its probabilistic cousin. Powerful and interpretable — but it can only draw straight boundaries, and it is only as good as the numbers you feed it.

Feature engineering (1990s–2011)

Before deep learning, computer vision meant humans designing features — SIFT, HOG, wavelets — that capture edges, corners, textures. A simple classifier (SVM, random forest) then decides. The intelligence lived in the features.

Takeaway In both, a human supplies the knowledge — the decision rule, or the features. The machine only fits the last step.

THE “DEEP” PARADIGM

Eras 3–4: let the machine learn the features

Deep learning (2012 →)

AlexNet won the ImageNet contest by a landslide using a deep convolutional network — features learned from raw pixels by stacking convolutions and training with backpropagation on GPUs. The lesson: stop engineering features, and learn them from data.

Foundation models (2017 →)

Transformers and self-supervised pretraining learn general-purpose representations from enormous unlabeled corpora, then adapt to new tasks with a handful of labels — the paradigm behind today’s large language and vision models.

Takeaway Here the knowledge moves into the architecture and the data — the human designs the learner, not the features.

THE SYNTHESIS

Where does human knowledge enter?

Era	Our model today	Where the human's knowledge enters
Linear models	Logistic regression	the decision rule — you pick the inputs, it draws a line
Hand-crafted features	Polar features + RF	the features — you encode the physics
Deep learning	CNN	the architecture — you design the inductive bias; features are learned
Foundation models	Advanced methods	the data — you curate pretraining; even the priors are learned

*Over sixty years, human knowledge migrated outward: rule → features → architecture → data.
These two lectures walk that entire path — on one wafer-map dataset, in about two hours.*

PART 1 · SECTION 1

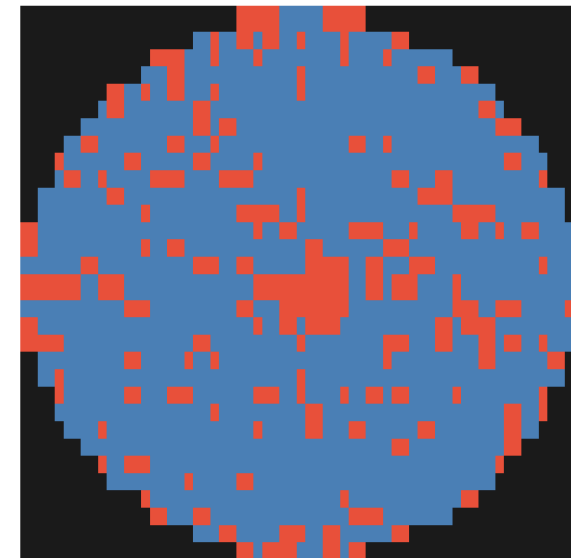
The problem

What are we classifying, and why is it not an image-recognition task?

MOTIVATION

Why wafer defect patterns matter

- A modern fab runs hundreds of process steps; every wafer holds hundreds to thousands of dies (chips).
- After probe test, each die is pass or fail. Plot the fails at their physical position and a spatial pattern often appears.
- The pattern is a fingerprint of the process fault: a ring points to edge effects, a center blob to spin/CMP nonuniformity, a thin line to handling damage.
- **Recognizing the pattern is the first step toward root cause — and toward yield, the metric a fab lives or dies by.**

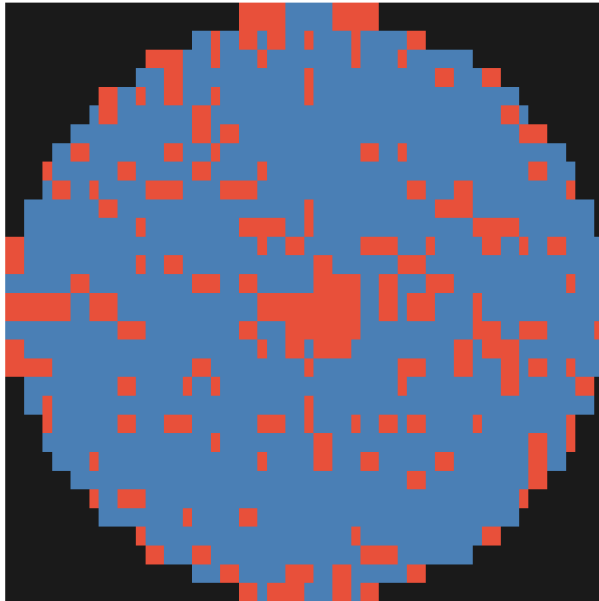


■ outside wafer ■ die passed ■ die failed

One wafer map: dies colored by probe-test outcome

DATA

What a wafer map actually is



■ outside wafer ■ die passed ■ die failed

- **A 2-D grid. Each cell is one die at its physical location on the wafer.**
- 0 = outside the wafer (grid is rectangular, wafer is circular → the black corners)
- 1 = die passed electrical probe test
- 2 = die failed
- Grid size varies by product: median 33×33 , ranging 25×22 to 175×155 .
- **The entire signal is the spatial arrangement of failing dies.**

DATA

A wafer map is not a photograph

No photographic statistics

- No texture, no color, no lighting, no natural-image edge distribution.
- Values are categorical (0/1/2), not an ordered intensity — 2 is not “more than” 1.
- So we encode the three states as separate channels, never as one number.

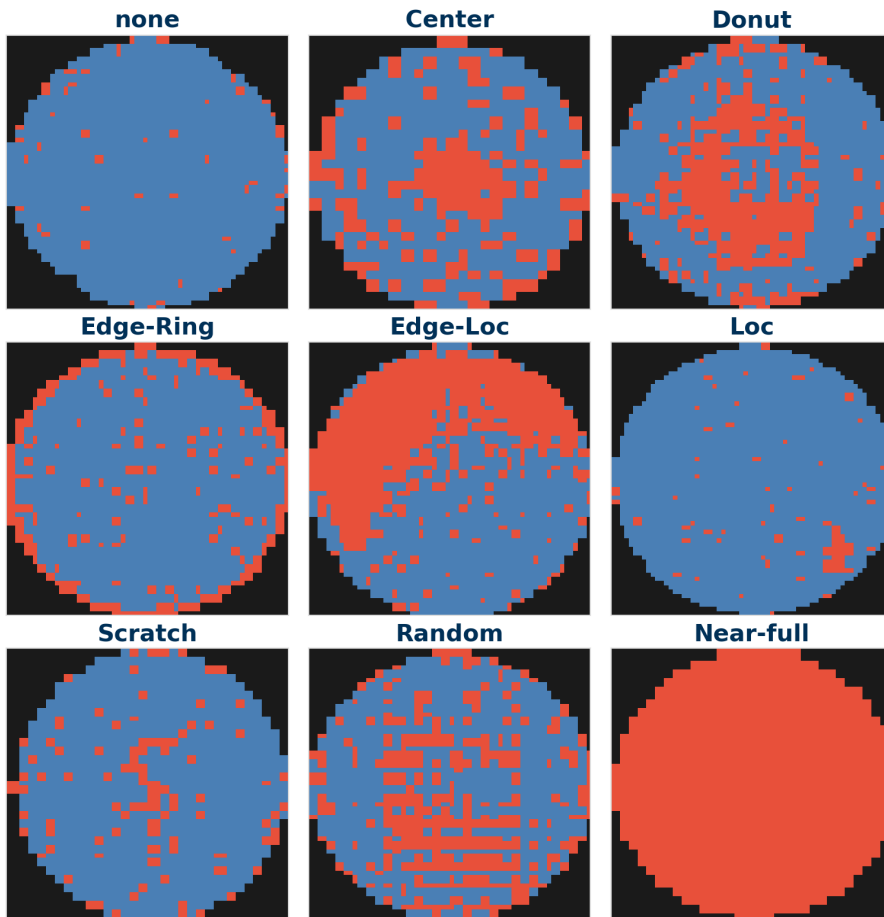
```
# categorical, not ordinal:  
# each state is its own channel  
torch.stack([x==0, x==1,  
             x==2], 1).float()
```

So pretrained vision models don't transfer

- DINOv2 and friends are trained on natural images — the statistics we just said don't exist here.
- Transfer learning from ImageNet buys little on wafer maps.
- Those models belong on real SEM / optical inspection images (DAGM, MVTec AD) — a different lab.
- Here, the right inductive bias comes from the geometry of the wafer, not from photographs.

DATA

Nine failure classes



- none — failures present, but no recognizable spatial pattern (85% of labels).
- **Center / Donut / Edge-Ring** — radially structured; a clean physical story.
- Edge-Loc / Loc — a compact blob, at the edge or anywhere.
- **Scratch** — a thin line from physical handling. Remember this one.
- Random — elevated but unstructured defectivity.
- Near-full — catastrophic excursion; the lot is scrap.

DATA

Each pattern has a physical origin

Pattern	Plausible process origin	Share
Edge-Ring	Edge bead, clamp / shadow ring, plasma edge effects	5.60%
Edge-Loc	Localized blob sitting at the wafer edge	3.00%
Center	Radial nonuniformity — spin coat, CMP, chuck temperature	2.48%
Loc	Compact blob anywhere — a particle or local defect	2.08%
Scratch	Thin line — physical contact, robot or chuck handling	0.69%
Random	Elevated but unstructured defectivity	0.50%
Donut	Annular — deposition or etch rate peaking at mid-radius	0.32%
Near-full	Catastrophic excursion — the lot is scrap	0.09%

Takeaway The label is a hypothesis about a physical cause — which is exactly why a physics-shaped model can compete with a CNN (Part 2).

PART 1 · SECTION 2

Data reality

Imbalance, label noise, and a leakage trap that quietly inflates every score.

DATA REALITY

The dataset: MIR-WM811K

811,457

real wafer maps from a production fab

21%

of wafers carry any label at all (172,950)

3%

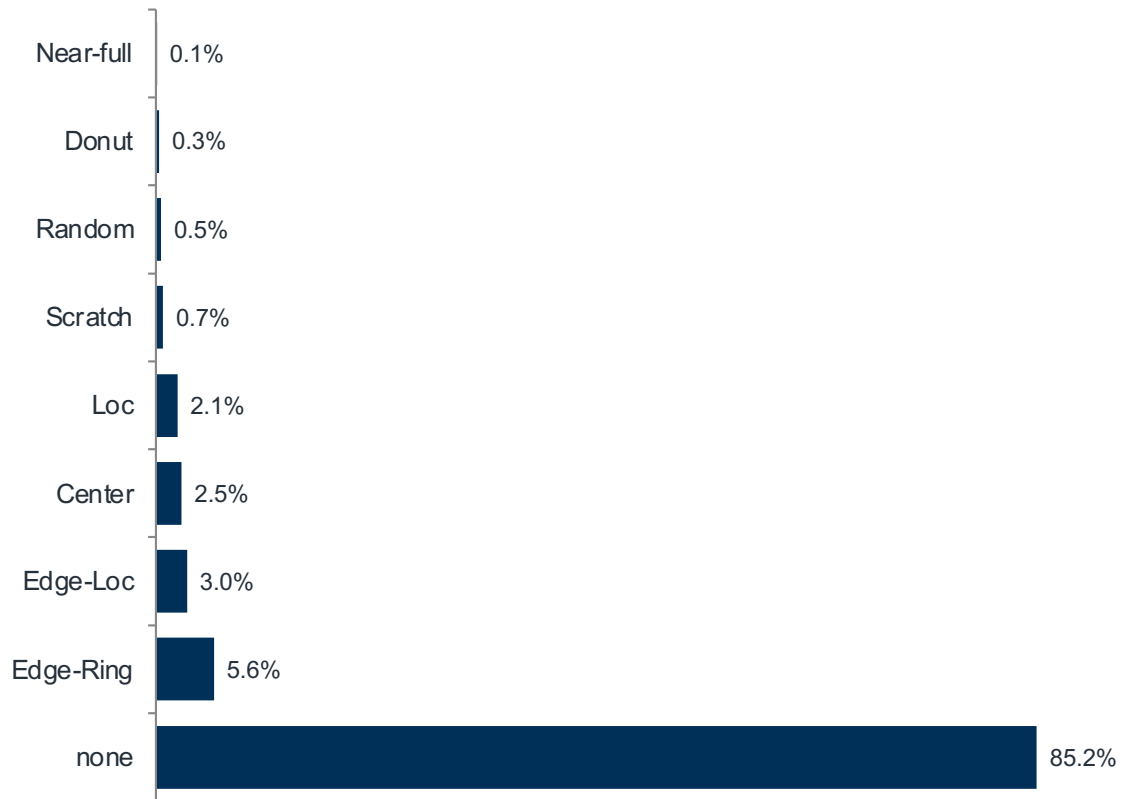
of the full set carry a named failure pattern

- Public, no login: mirlab.org — ships a Python pickle and a MATLAB v7.3 (HDF5) copy.
- We hand students .npz, never the 1.9 GB pickle: pickle load is 15+ min, needs several GB, and is unsafe (arbitrary code execution on load).
- Wafers stay uint8; states are expanded to channels per batch (~50× memory saving).

License requires citation: Wu, Jang & Chen, "Wafer Map Failure Pattern Recognition and Similarity Ranking for Large-Scale Data Sets," *IEEE Trans. Semiconductor Manufacturing*, 28(1), 2015. doi:10.1109/TSM.2014.2364237

DATA REALITY

Severe, structural class imbalance



- One class (none) is 85% of every labeled batch.
- Near-full has 149 examples in the entire dataset.
- A model that always predicts none scores 85% accuracy and learns nothing.
- This shapes every choice ahead: the loss, the metric, the split.

“none” is not clean — and labels are noisy

What “none” really means

- 9.1% of on-wafer dies fail — including inside none wafers.
- none means no recognizable pattern, not a clean wafer.
- It is baseline yield loss with no spatial signature — a real class, not an absence.

A real label-noise ceiling

- Labels came from human engineers, not confirmed root-cause investigations.
- Loc, Edge-Loc, and Scratch overlap genuinely — labelers disagreed.
- No model can exceed the consistency of its labels. Part 2 estimates this floor empirically.

Takeaway Expect an accuracy ceiling below 100% that no architecture can beat — because the labels themselves disagree.

Train, development, and test sets

Train 70%

Dev 15%

Test 15%

70% Training set

The model learns its weights from these wafers — this is the data it fits.

15% Development set

Used to compare models and tune choices, without touching test. Also called the validation set.

15% Test set

Locked away until the end. Measures performance on wafers the model has never seen, and is reported once.

Think of it as your study notes, a practice exam, and the final exam — you never study from the final.

Takeaway The test set never influences training or tuning. That is the only way its score reflects performance on genuinely new wafers.

The leakage trap: split by lot, not by wafer

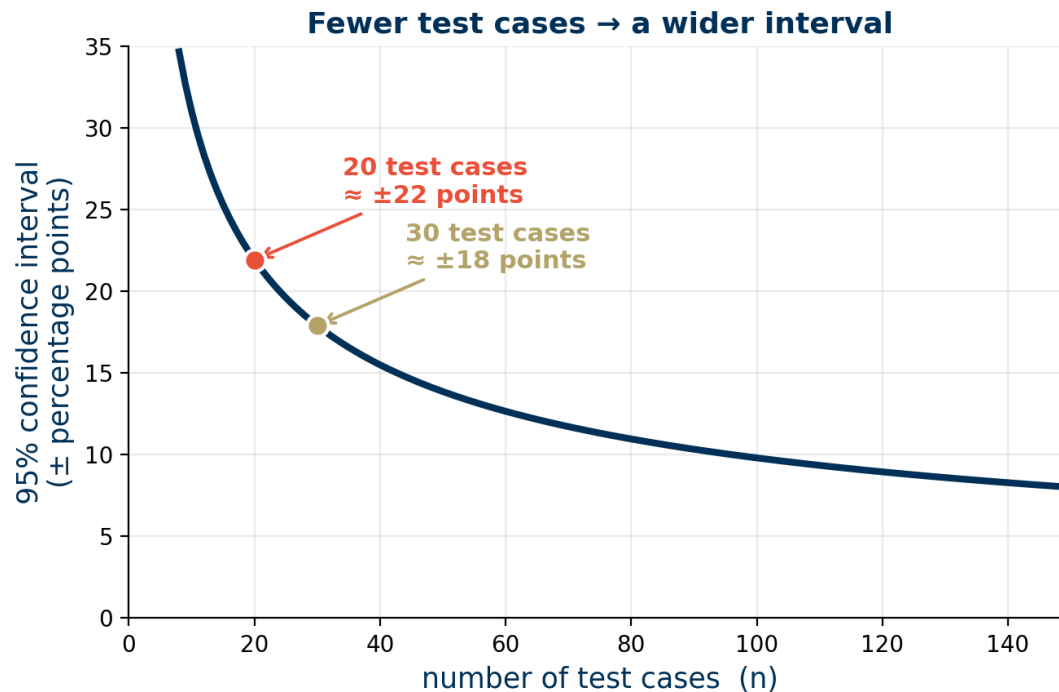
- Wafers from one lot ran through the same tools in the same time window — they are near-duplicates.
- A random wafer-level split scatters siblings across train and test.
- **The model then “recognizes” test wafers it effectively trained on, and the score is inflated.**
- **Fix: group by lot so no lot appears in two splits. Verified: zero lots shared between any pair.**

```
from sklearn.model_selection \
    import GroupShuffleSplit

# the one line that
# stops the leak:
split(X, y, groups=lot)
```

Takeaway Dev and test scores agree closely all lecture — evidence the grouped split is working and we are not overfitting to dev.

Confidence intervals: how much to trust a number



- A recall measured on only twenty wafers is an estimate, not the exact truth.
- Draw a different twenty wafers and the number would move.
- A ninety-five percent confidence interval is the range that would contain the true value in ninety-five out of a hundred such draws.
- Fewer test cases make that interval wider. With twenty Near-full wafers, it is about plus or minus twenty-two points.

$$\text{half-width} \approx 1.96 \times \sqrt{(p(1-p) / n)}$$

DATA REALITY

Split design: 70 / 15 / 15, grouped by lot

Split	Wafers	Lots	Near-full
train	121,061	7,546	108
dev	25,956	1,618	21
test	25,933	1,598	20

- Rare classes deliberately balanced across splits (rare-class-heavy lots dealt first).
- The binding constraint is Near-full, not the total: with 20 test cases, the 95% CI on recall is $\approx \pm 22$ points.
- Why not 60/20/20? That convention is from an era of small datasets. More test cases narrow the CI but steal training examples from a class with only 149 in existence.
- **Decision: keep 70/15/15 and have students report confidence intervals — turning the limit into a lesson.**

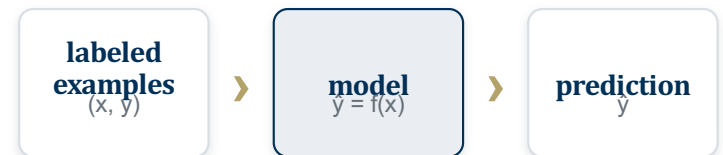
PART 1 · SECTION 3

Learning from data: the linear baseline

What machine learning is, logistic regression from first principles, and the metric trap.

What is machine learning?

- **Instead of writing rules by hand, we show the computer many labeled examples and let it find a rule that fits.**
- Example x — a wafer map. Label y — its class (Center, Scratch, ...).
- A model f turns an input into a prediction: $\hat{y} = f(x)$. A loss scores how wrong it is.
- We fit the model on a training set, then measure it on data it has never seen.
- **The goal is generalization, not memorization — doing well on new wafers, not the ones it studied.**

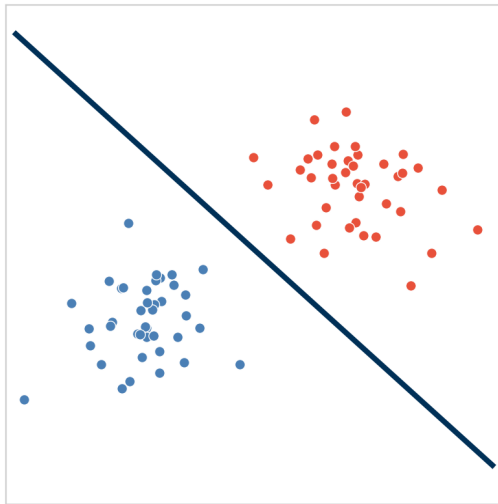


Learning loop: predict → measure the loss → nudge the model to reduce it → repeat.

Takeaway Every model today does the same thing — fit labeled examples, then generalize. They differ only in the shape of f .

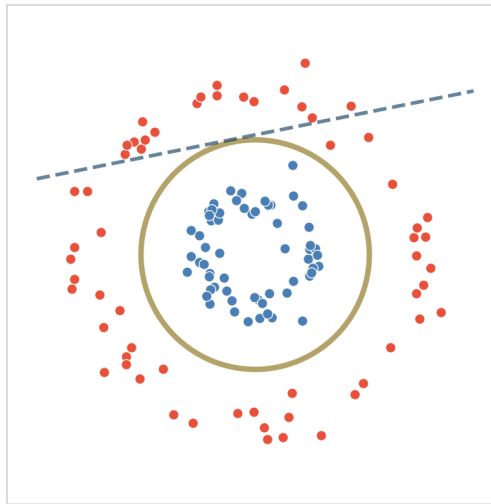
Classification means carving up a space

A straight line works



linear boundary

A straight line fails



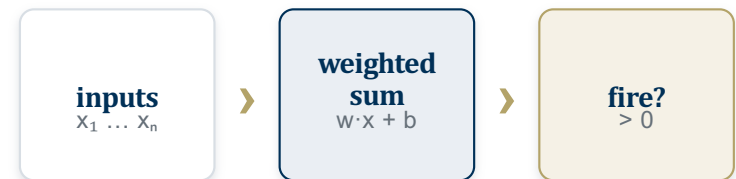
needs a curved boundary

- Turn each example into a point whose coordinates are its features.
- **A classifier splits that space into regions — one per class.**
- The surface between regions is the decision boundary; learning means finding it from the labeled points.
- **A linear model can only place straight boundaries. Curved patterns need a richer model.**

Takeaway Keep this picture in mind: every model we build is really choosing the shape of the boundaries it can draw.

One neuron: the perceptron (1958)

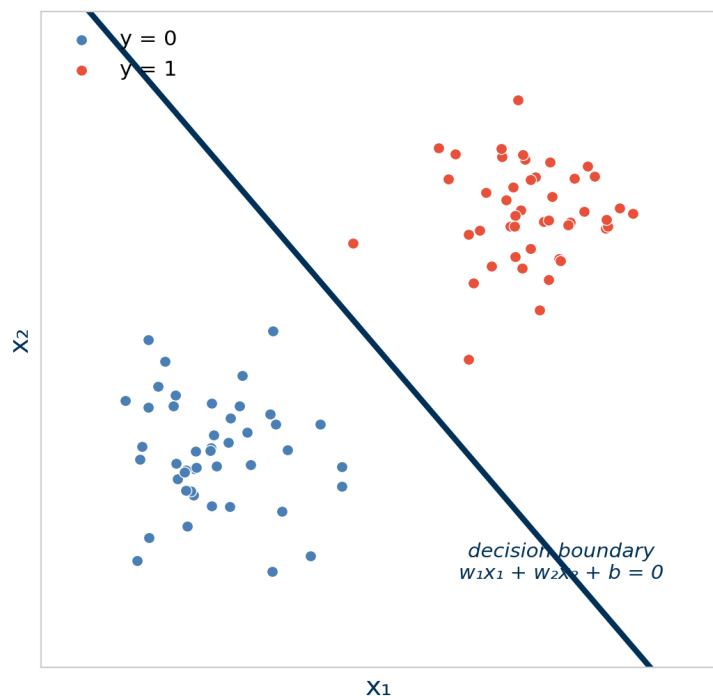
- The original learnable unit: multiply each input by a weight, add them up, add a bias.
- If the total clears a threshold, it fires (class 1); otherwise it doesn't (class 0).
- **It learns by nudging weights whenever it makes a mistake — the seed of everything that followed.**
- **But one perceptron can only separate with a straight line — the limit that stalled AI for years (Minsky & Papert, 1969).**



```
output = 1 if (w · x + b) > 0, else 0
```

Takeaway A weighted vote of the inputs. Logistic regression is this idea, made smooth and probabilistic.

Logistic regression: the model



Two features (x_1, x_2) — the two axes. Each point is one example, colored by its label $y \in \{0, 1\}$.

The model, in four steps

1. Linear score

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + b$$

2. Squash to a probability

$$h = g(z) \quad (\text{defined next slide})$$

3. Predict the class

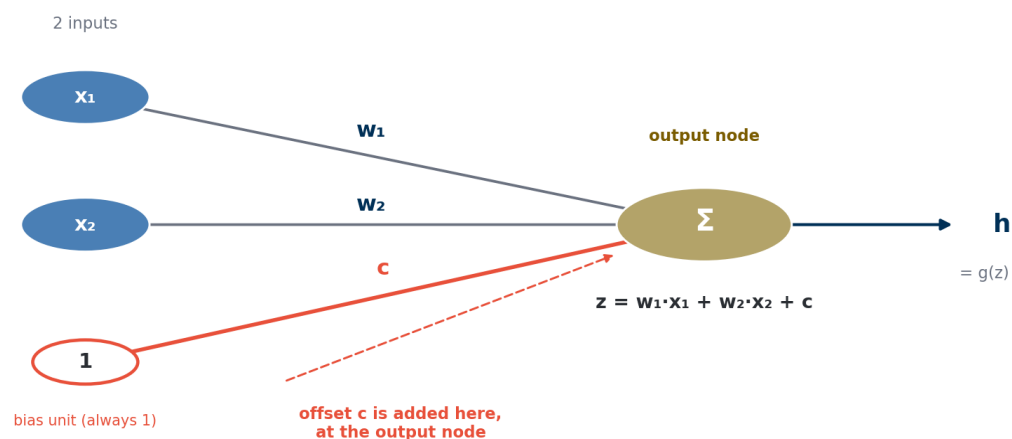
$$\hat{y} = 1 \text{ if } h \geq 0.5, \text{ else } 0$$

4. Decision boundary

$$z = 0 \rightarrow \text{the straight line at left}$$

w_1, w_2, b are the numbers the model learns.

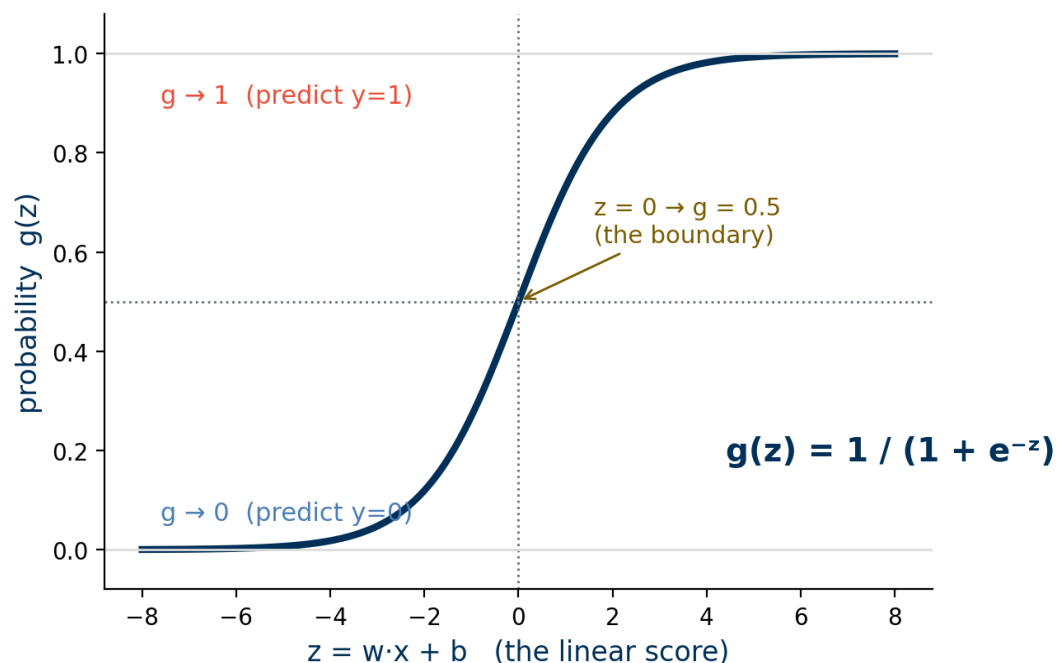
The same model, drawn as a network



- Logistic regression is the simplest network there is: the two inputs connect straight to a single output node — no hidden layer at all.
- Each input carries a weight (w_1, w_2), and the output node adds up the weighted inputs.
- The offset c — the bias b from the last slide — is added at the output node itself. It rides in on a constant “1” input, so it shifts the score without being tied to any x .
- That output node then squashes the sum with the sigmoid to give the probability h . This single node is logistic regression.

Takeaway One output node, three learned numbers — w_1 , w_2 , and the offset c . Inserting nodes between input and output is exactly what will turn this into a deep network.

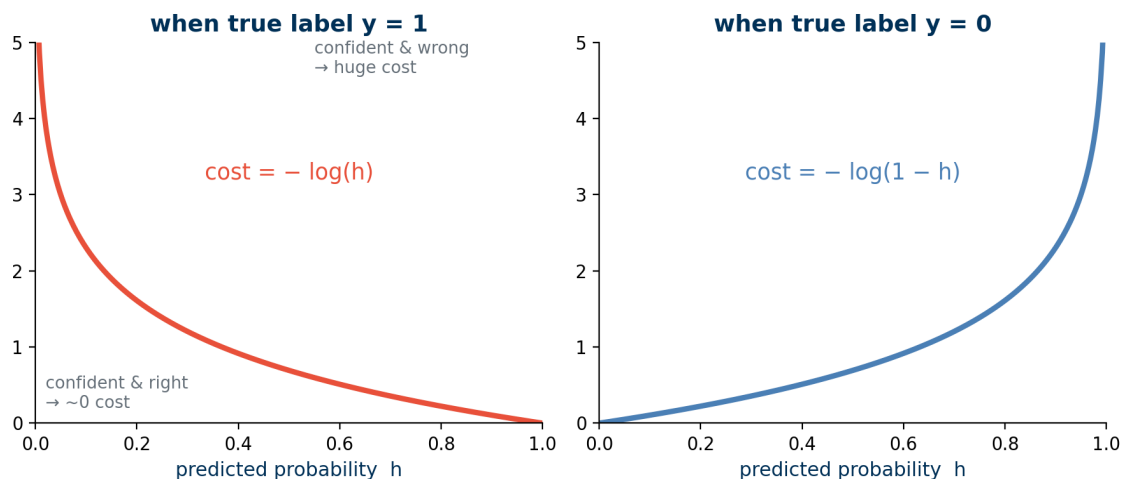
The nonlinearity: the sigmoid



- The score z can be any real number — but we need a probability between 0 and 1.
- **The sigmoid (logistic) function does exactly that squashing — the smooth S-curve.**
- $h = g(z)$ is read as the probability that the label is 1.
- Big positive $z \rightarrow h$ near 1; big negative $z \rightarrow h$ near 0; $z = 0 \rightarrow h = 0.5$, the boundary.
- **This smooth squashing is the nonlinearity — it turns a hard yes/no into a soft probability.**

$$h = g(z) = 1 / (1 + e^{(-z)}) = P(y = 1 \mid \mathbf{x})$$

How wrong are we? The cost function

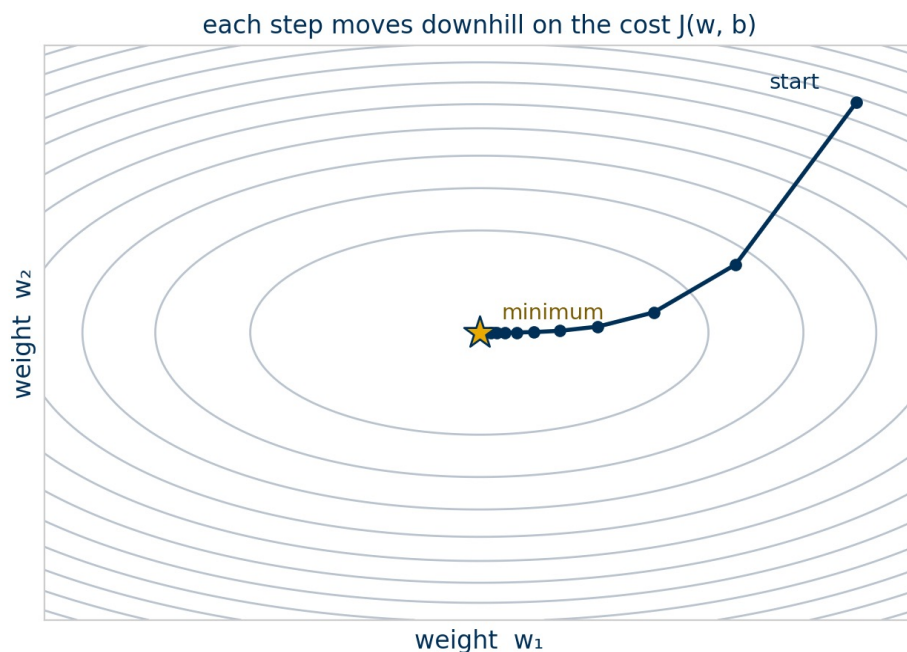


- We want the predicted probability h to match the true label y .
- If $y = 1$, penalize by $-\log(h)$; if $y = 0$, penalize by $-\log(1-h)$.
- Confident and wrong $\rightarrow$ the cost shoots toward infinity. Confident and right $\rightarrow$ cost near 0.
- One formula covers both cases at once.

one example: $\text{cost} = -[y \cdot \log(h) + (1-y) \cdot \log(1-h)]$
 all m examples: $J(w, b) = -(1/m) \sum [y \cdot \log(h) + (1-y) \cdot \log(1-h)]$

Takeaway This average is the cross-entropy (log-loss). Training means making J as small as possible.

Minimizing the cost: gradient descent



- **We want the w, b that make the cost J smallest.**
- J is bowl-shaped, so we walk downhill: step in the direction that lowers J , then repeat.
- α is the learning rate — how big each step is.
- **The gradient works out to a strikingly simple form.**

The weight-update rules

$$w_j \leftarrow w_j - \alpha \cdot (1/m) \sum (h - y) \cdot x_j$$

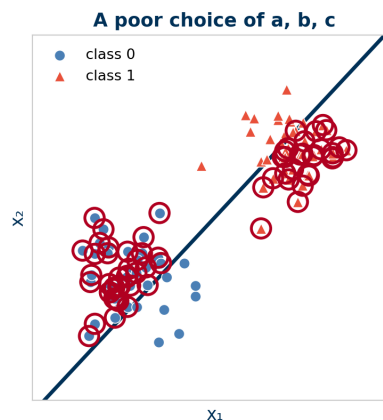
$$b \leftarrow b - \alpha \cdot (1/m) \sum (h - y)$$

$(h - y)$ is just “prediction minus truth.”

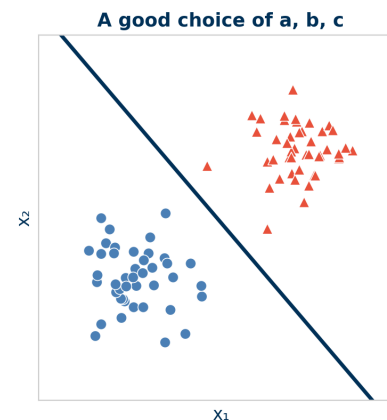
The whole problem: choose a , b , c

`prediction = sigmoid( f(x) ),       $f(x) = a \cdot x_1 + b \cdot x_2 + c$`
 goal: choose the three numbers a , b , c so that the data are classified

- The line $f(x) = 0$ splits the plane into two sides.
- Where $f(x)$ is positive, the sigmoid gives a probability above one-half $\rightarrow$ class 1; where it is negative $\rightarrow$ class 0.
- “Classified” means each point sits on its own side of the line.
- a and b are the two weights (w_1 , w_2); c is the bias.
 a and b tilt the line, c shifts it.



loss $J = 1.05$ · 66 of 90 on the wrong side



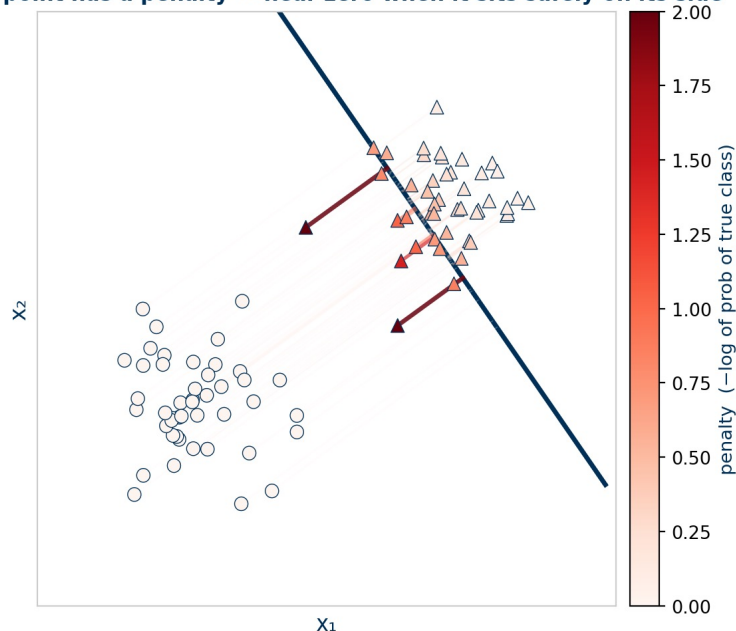
loss $J = 0.01$ · 0 of 90 on the wrong side

The loss J adds a penalty for every point — big when it sits on the wrong side of the line, near zero when it sits confidently on the right side.

Takeaway Learning is the search for the a , b , c that make the loss J smallest — the boundary that classifies the data. The next slide runs that search.

The loss J: a penalty for every point

Every point has a penalty — near zero when it sits safely on its side



$$J = - \sum [y \cdot \log(h) + (1-y) \cdot \log(1-h)]$$

$h = \text{sigmoid}(a \cdot x_1 + b \cdot x_2 + c)$, summed over all wafers

- Every point contributes a penalty: the $-\log$ of the probability the model gives to that point's true class.
- **A point safely on its side gets a probability near 1, so its penalty is near zero — faint on the left.**
- **A point on the wrong side gets a small probability, so $-\log$ of it is large — the red points.**
- J adds these over all the wafers. Small J means almost every point sits on the correct side.

Takeaway Every point counts, but the wrong-side points dominate J. Training moves the line to turn the red points faint.

The update rule for a, b, c

- We now have the loss J. Learning means nudging a, b, and c to make J smaller — one small step at a time.
- Each step subtracts a slice of the error. On one wafer the error is $(h - y)$: the predicted probability minus the true label, 0 or 1.
- For a and b, weight that error by the input it multiplies — x_1 for a, x_2 for b. For the bias c there is no input, so it is just the error itself.
- **α is the learning rate — how big each step is; the sum averages over all m wafers.**

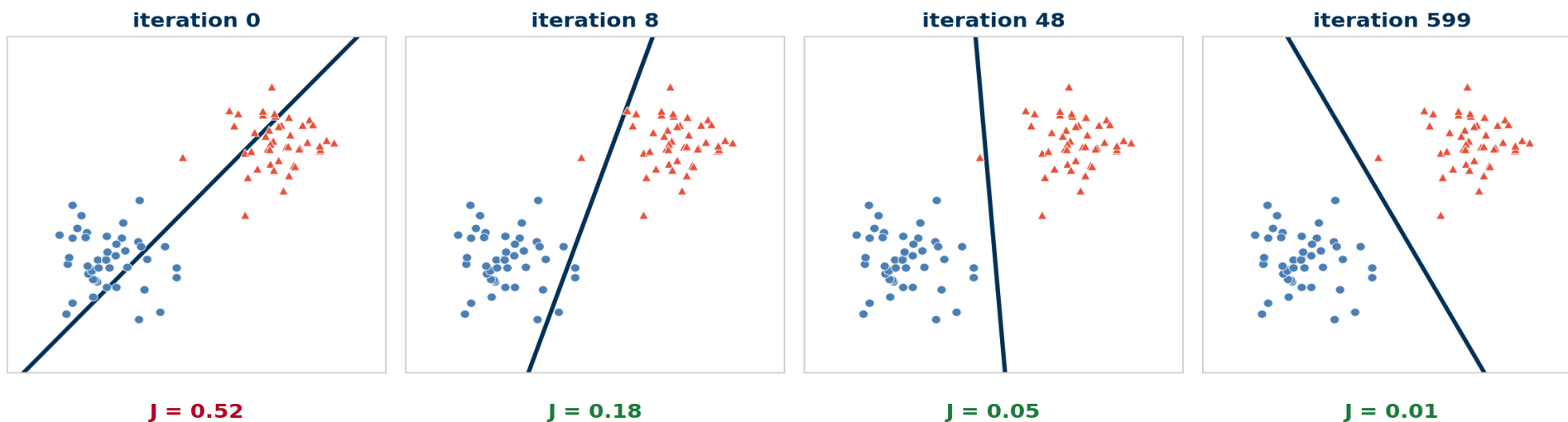
One step downhill, on all three numbers

$$\begin{aligned} a &\leftarrow a - \alpha \cdot (1/m) \sum (h - y) \cdot x_1 \\ b &\leftarrow b - \alpha \cdot (1/m) \sum (h - y) \cdot x_2 \\ c &\leftarrow c - \alpha \cdot (1/m) \sum (h - y) \end{aligned}$$

In words: move each of a, b, c in the direction that lowers J, by an amount set by how much that number is to blame for the error $(h - y)$.

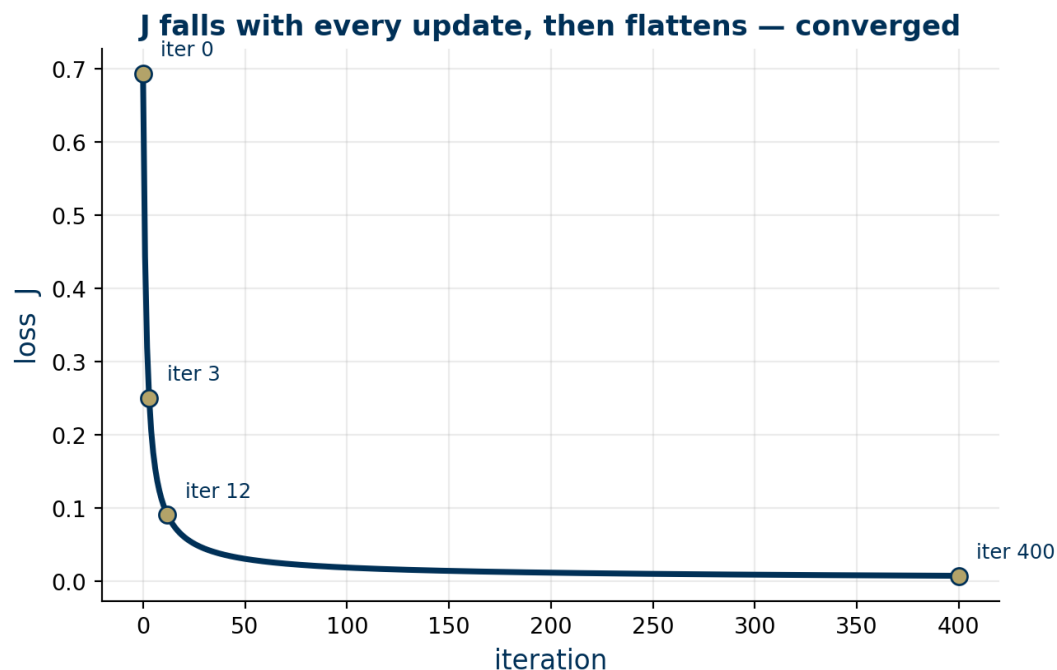
Takeaway This is the same gradient-descent step as before, written for the a, b, c we chose — three numbers, each corrected by (prediction – truth).

The line evolves with the iterations



- At iteration 0 the line starts badly oriented — it runs across both clusters — so the loss J is high, about 0.52.
- Each update nudges a , b , c ; the line rotates toward the gap between the clusters, and J falls — 0.18, then 0.05.
- By a few hundred iterations the line has swung around to cleanly separate the two clusters, and J has flattened near 0.01.

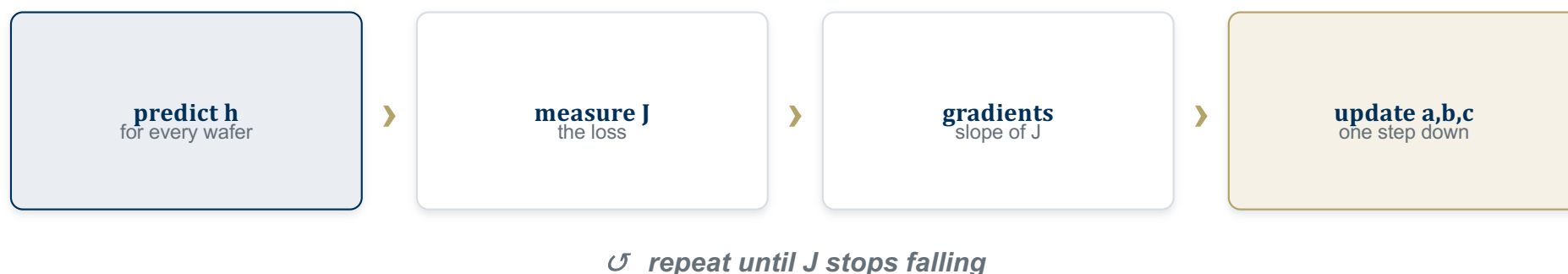
J falls until it converges



- Every iteration lowers J by one gradient-descent step on a, b, c.
- J drops steeply at first, while the line is far from right, then slows as it settles into place.
- **When J stops falling, the model has converged — it has found the best a, b, c it can, and more iterations would not help.**

$$a, b, c \leftarrow a, b, c - \alpha \cdot (\text{gradient of } J)$$

The training loop, in four steps

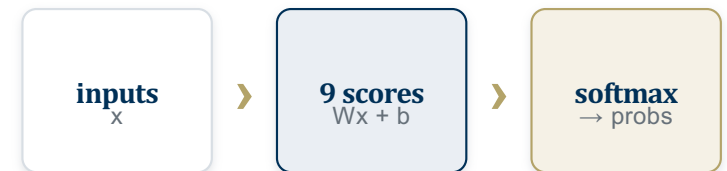


- Start with a, b, c at zero, then run the four steps above over and over.
- **Each pass rotates and shifts the line a little, and lowers J .**
- **Stop when J stops dropping — the model has converged.**

Takeaway This exact loop — predict, measure the loss, take a gradient step, repeat — is how we train the CNN too. Only the model f changes.

From two classes to nine: the softmax

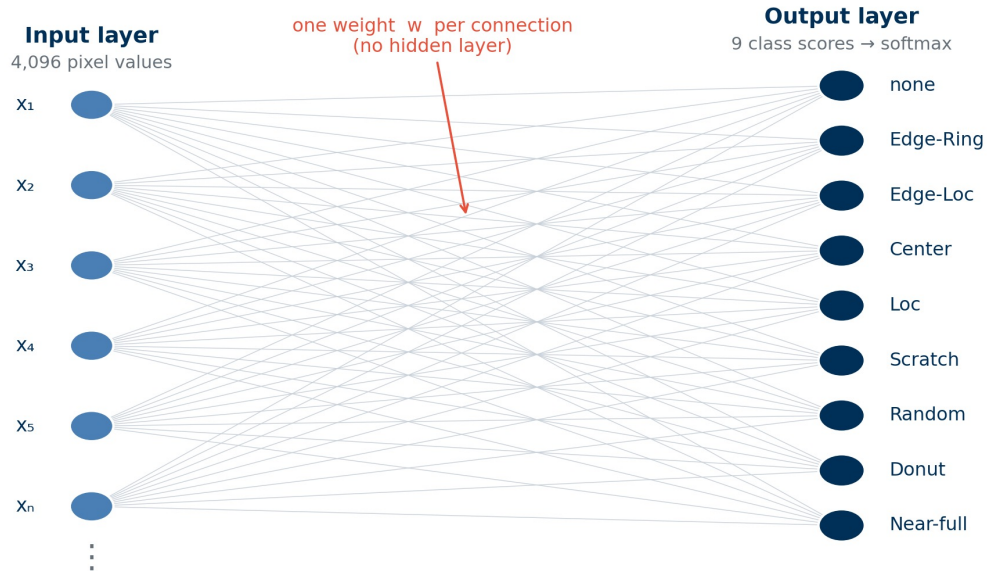
- Our wafer problem has 9 classes, not 2 — so we need one score per class, not a single yes/no.
- Compute a score for each class (its own weights), then convert all nine scores to probabilities that sum to 1 with the softmax — the multiclass sigmoid.
- Predict the highest-probability class. Train with the same cross-entropy loss and gradient descent.
- Everything else carries over unchanged — including the key limitation.
- **Key point: still one weight per input feature, per class.**



$$p(\text{class } c \mid \mathbf{x}) = \text{softmax}(\mathbf{w}_c \cdot \mathbf{x} + b_c)$$

Takeaway One weight per feature is the whole assumption: each input matters on its own, independent of its neighbors.

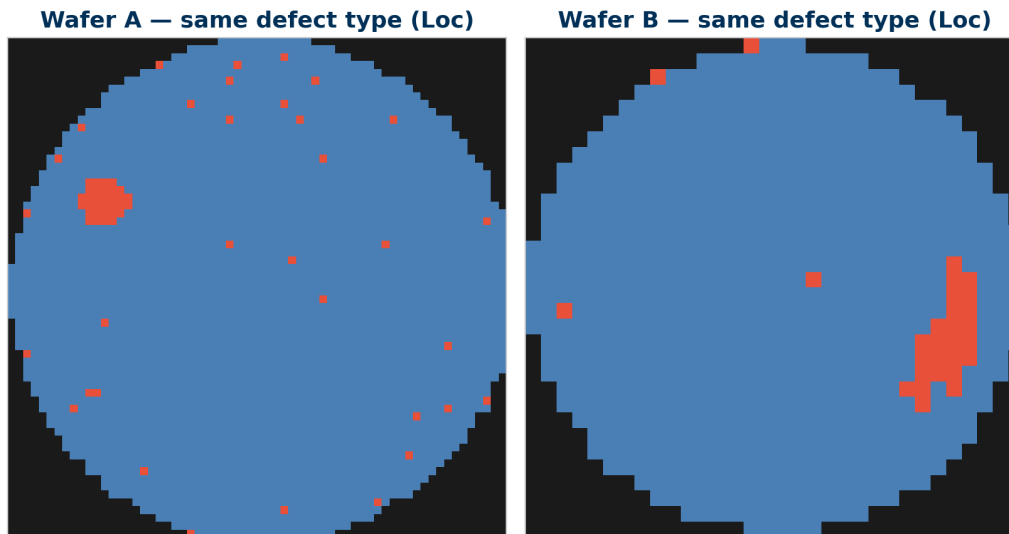
Logistic regression is a one-layer network



- Draw it as a network: every input pixel connects straight to every class output — with no hidden layer in between.
- Each connection carries one weight; each output is a weighted sum, squashed by softmax.
- That is the entire model — a single layer of weights. Fast, interpretable, and linear.
- The missing hidden layers are exactly why it can only draw straight-line boundaries.

Takeaway A CNN is what you get when you insert clever, spatially-aware layers between input and output — the subject of the next section.

The blind spot on images



Same defect type (Loc), two positions → two unrelated input vectors to a linear model.

- To feed a wafer map to logistic regression, we flatten it into 4,096 pixel values — one weight per pixel.
- **The same defect in a new position lights up different pixels — so the model sees a completely different input.**
- It has no idea pixel (i,j) sits next to $(i,j+1)$, and no way to recognize a pattern that has moved.
- **This is the assumption we will watch break — and the reason we need convolutions.**

Precision, recall, F1, and macro-F1

	Predicted: Scratch	Predicted: not Scratch	
Actually Scratch	True Positive <i>caught</i>	False Negative <i>missed</i>	Recall = $TP / (TP + FN)$ of the true Scratches, how many caught
Actually not Scratch	False Positive <i>false alarm</i>	True Negative <i>correct reject</i>	

Precision = $TP / (TP + FP)$
of the alarms, how many were right

F1 — one number from the two

$$F1 = 2 \cdot P \cdot R / (P + R)$$

- The harmonic mean of precision and recall — it stays low unless both are high.
- Example: precision 1.00, recall 0.02 → the ordinary average is 0.51, but $F1 \approx 0.04$.

Macro-F1 — one number across nine classes

The average of all nine per-class F1 scores, each class weighted equally — so a rare class like Scratch counts for exactly as much as the common “none.”

n (in the per-class tables) = the number of test wafers of that class.

Takeaway Accuracy can be high while macro-F1 stays near zero. Macro-F1 is the number that refuses to ignore the rare classes.

What a good F1 and a bad F1 look like

GOOD Center, from the CNN

	flagged Center	flagged not	
really Center	606	38	Recall = $TP / (TP + FN)$
really not	185	25,104	

$$\text{Precision} = TP / (TP + FP)$$

P = 0.77 R = 0.94

F1 = 0.85

Catches 606 of the 644 center wafers; about a quarter of its alarms are false. Both scores stay high — a detector worth trusting.

BAD Scratch, from the polar model (next lecture)

	flagged Scratch	flagged not	
really Scratch	4	206	Recall = $TP / (TP + FN)$
really not	0	25,723	

$$\text{Precision} = TP / (TP + FP)$$

P = 1.00 R = 0.02

F1 = 0.04

Right every time it fires — but it fires 4 times all shift and misses 206 scratches. Useless as an alarm.

What you want: F1 as close to 1 as possible. It climbs toward 1 only when precision and recall are both high — a detector that catches the real defects without crying wolf.

BASELINE

First, the baseline everyone forgets

0.854

accuracy — always predict “none”

$\approx$ 0.10

macro F1 of that same model

- Predicting the majority class gets 85% accuracy while detecting nothing.
- This is the number any real model must beat on a metric that counts the rare classes.
- Macro F1 averages per-class F1 with equal weight — so ignoring 8 of 9 classes is punished, hard.

Takeaway Report accuracy and macro F1 together, always. One hides the imbalance; the other exposes it.

BASELINE

Logistic regression on flattened pixels

- Flatten each 64×64 map to a 4,096-dim vector. Every pixel is an independent feature.
- **That is the model's core assumption — and it has no notion of where a pixel sits.**
- `class_weight="balanced"` re-weights the loss so the 147k none wafers stop dominating.
- Trains on all 121,061 wafers in about 1m 22s.

```
F = X.reshape(len(X), -1)
F = F.astype('float32') / 2

clf = LogisticRegression(
    max_iter=400,
    class_weight='balanced')
clf.fit(F, y)
```

4,096 features · no spatial structure

The result nobody expects

accuracy 0.688 < 0.854 majority baseline

Worse by the metric everyone reaches for first — and better by every metric that matters.

0.688

accuracy (test)

0.473

macro F1 (test)

8 / 9

classes now actually detected

Takeaway `class_weight="balanced"` is a choice, not a default. It trades headline accuracy for the ability to see the rare classes at all.

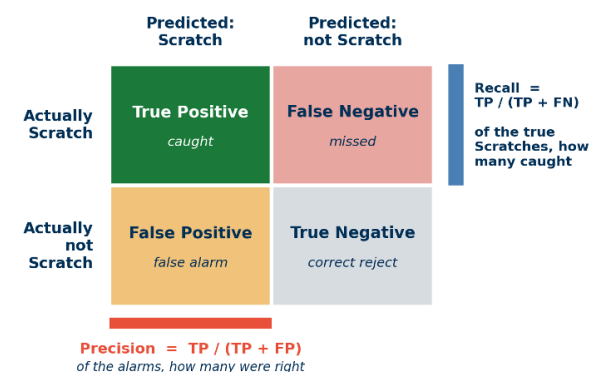
BASELINE

Where logistic regression wins and loses

Class	Precision	Recall	F1	n
none	0.976	0.678	0.800	22,224
Edge-Ring	0.928	0.967	0.947	1,378
Edge-Loc	0.204	0.671	0.313	706
Center	0.690	0.936	0.794	644
Loc	0.059	0.366	0.102	543
Scratch	0.029	0.300	0.053	210
Random	0.234	0.368	0.286	125
Donut	0.718	0.614	0.662	83

- Global, radial signatures survive flattening — Edge-Ring F1 0.95.
- Small blobs whose position varies collapse — Loc, Scratch near-zero precision.

- Scratch precision 0.029: for every true scratch it flags, ~34 are false alarms.
- Same model, same training. The difference is pure geometry.



THE ARGUMENT FOR CNNs

One model, two fates — and why

0.967

Edge-Ring recall

global radial signature

vs

0.366

Loc recall

small blob, variable position

- Edge-Ring looks the same after flattening — its failing dies land in the same feature indices every time.
- Loc is a blob that can sit anywhere. Flattening ties each feature to a fixed pixel, so a shifted blob is a different input.

The assumption that broke: “pixels are independent features.” A linear model on raw pixels has **zero translation invariance**. To recognize a pattern wherever it appears, the model needs to know that nearby pixels belong together — that is exactly what a convolution builds in. The argument for CNNs, derived from the data rather than asserted.

PART 1 · SECTION 4

The deep neural network

Logistic regression, stacked into layers — until the straight-line boundary starts to bend.

A short history of neural networks

Perceptron

Rosenblatt builds a single trainable neuron — essentially logistic regression in hardware. Great early excitement about machines that learn.

Minsky & Papert

They prove a single perceptron cannot even learn XOR. Funding dries up — the first “AI winter.”

Backpropagation

Rumelhart, Hinton & Williams give an efficient way to train many stacked layers at once. Multi-layer networks become practical.

Deep learning

More data, GPUs, and better training make very deep networks work. AlexNet on ImageNet ignites the modern boom.

1958

1969

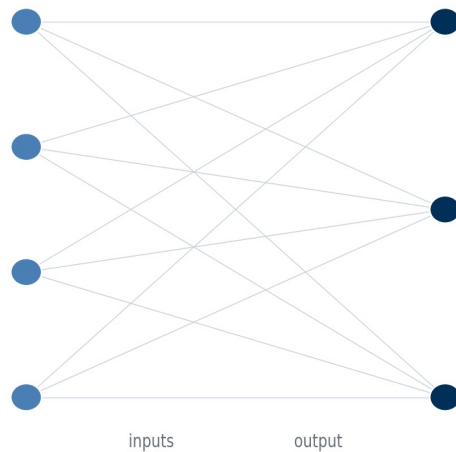
1986

2012

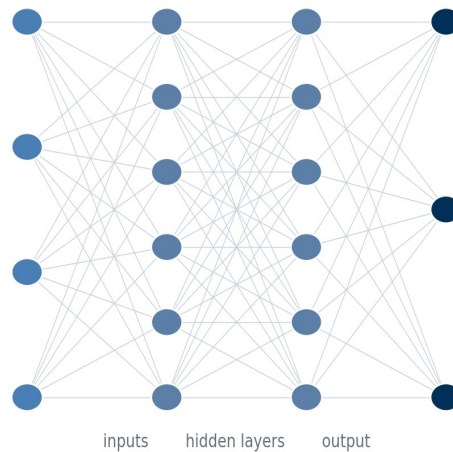
Takeaway One neuron is logistic regression. The whole story of deep learning is learning to train many of them, stacked in layers.

A deep network is logistic regression, stacked

Logistic regression
one layer: inputs → output



Deep neural network
inputs → hidden → hidden → output



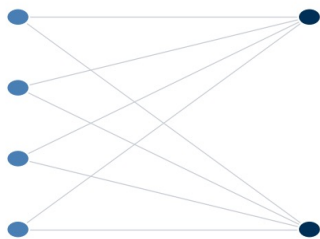
- Logistic regression sends the inputs straight to the output through one layer of weights — one weighted sum, one sigmoid.
- **A deep network inserts hidden layers in between. Each neuron is its own little logistic unit: a weighted sum, then a nonlinearity.**
- The nonlinearity is ReLU — keep positive values, set negatives to zero. It is what lets stacked layers do more than one big linear step.
- **More layers let the network compose simple pieces into complex shapes. That stacking is what “deep” means.**

each layer: $\text{ReLU}(W \cdot x + b)$, $\text{ReLU}(z) = \max(0, z)$

Takeaway Same building block as logistic regression — just many of them, wired in layers.

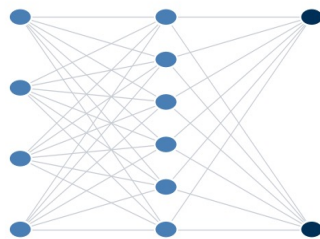
What makes a network “deep”?

0 hidden layers



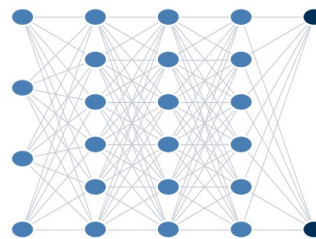
logistic regression (linear)

1 hidden layer



a shallow neural network

2 or more hidden layers



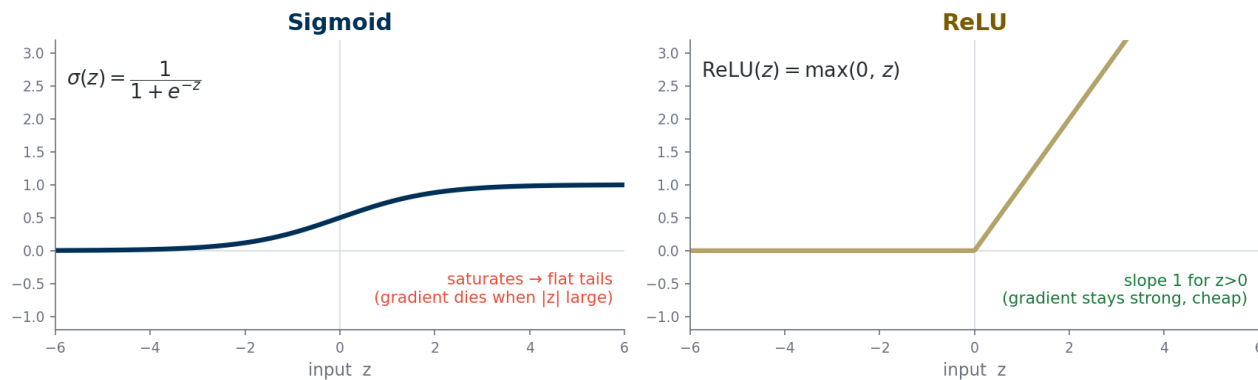
a deep neural network (DNN)

- “Deep” refers to depth — the number of hidden layers, the layers between the input and the output.
- Zero hidden layers is logistic regression: inputs go straight to outputs, and the boundary is a straight line.
- One hidden layer is already a neural network — the multilayer perceptron — but it is usually called shallow, not deep.
- **Two or more hidden layers is what people mean by a deep network. There is no exact threshold; the common convention is “more than one hidden layer.”**

Why go deep at all? one hidden layer can, in theory, approximate any function (the universal approximation theorem), but it may need an enormous number of neurons. Adding layers builds features on top of features, reaching the same accuracy far more efficiently.

Takeaway Depth is just the count of hidden layers. One is a neural network; two or more earns the name “deep.”

The activation: ReLU vs the sigmoid

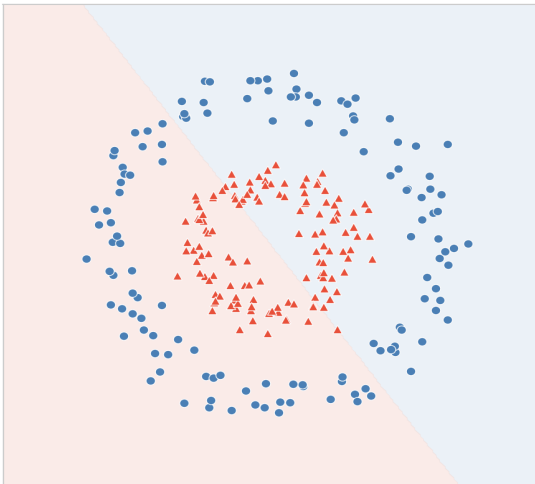


- **Both are nonlinearities — the kink that lets stacked layers bend boundaries. Without one, the layers collapse to a straight line.**
- The sigmoid squashes any input into (0, 1). It is the right shape for a final probability, but inside a deep net its tails go flat, so the gradient there is nearly zero and learning stalls.
- **ReLU keeps the value if positive and zeroes it otherwise. Its slope stays a constant 1 for positive inputs, so gradients flow through many layers, and it is trivially cheap to compute.**

Takeaway Rule of thumb: ReLU for the hidden layers so gradients survive the depth; a sigmoid (or softmax) only at the output, to read out a probability.

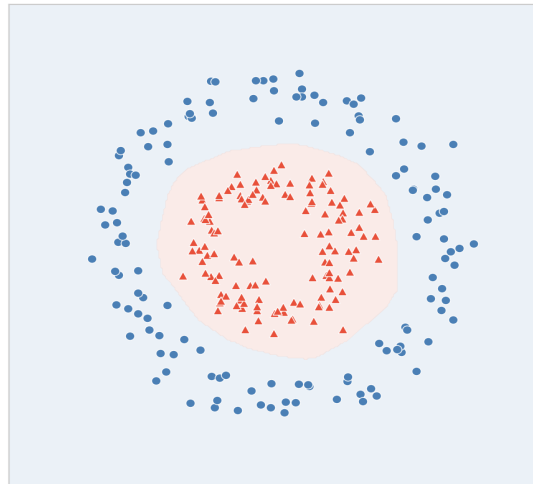
A deep network bends the boundary

Logistic regression · one straight line



accuracy 0.49

A deep network (DNN) · a curved boundary



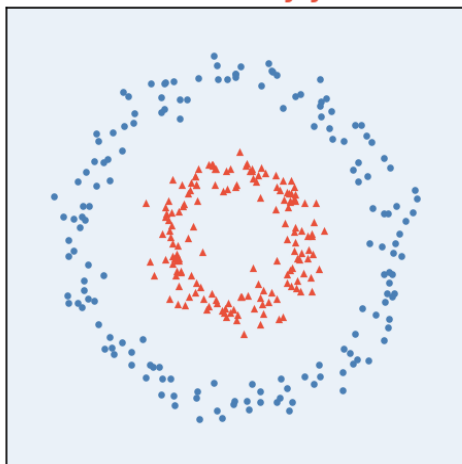
accuracy 1.00

- Here the two classes are one ring inside another. A straight line cannot separate them, so logistic regression scores about 0.49 — a coin flip.
- **Stack two hidden layers with ReLU and the network bends the boundary into a curve, wrapping the inner ring perfectly — accuracy 1.00.**
- **That is the payoff of depth: more layers give more flexible shapes than a single straight line ever could.**
- The same lesson our failing Loc and Scratch classes were asking for — a boundary that is not a straight line.

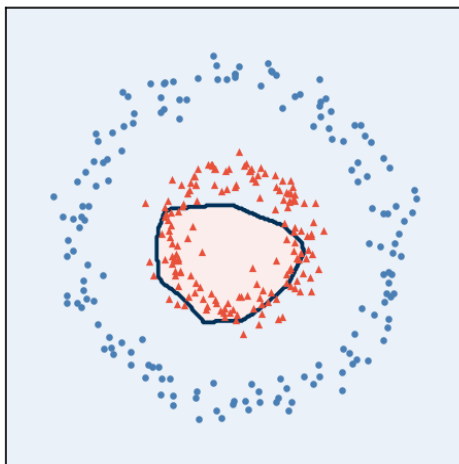
Takeaway Depth buys flexible boundaries. Next: how a network with many layers is actually trained.

The boundary bends as it trains

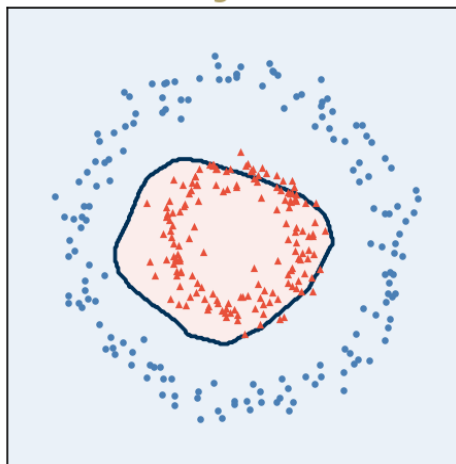
iteration 3 · acc 0.50
no boundary yet



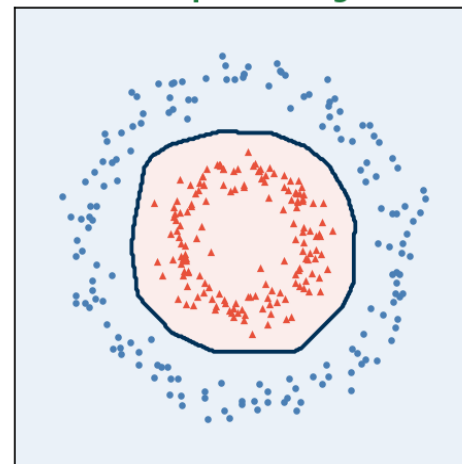
iteration 12 · acc 0.72
a small curve forms



iteration 20 · acc 0.94
bending outward

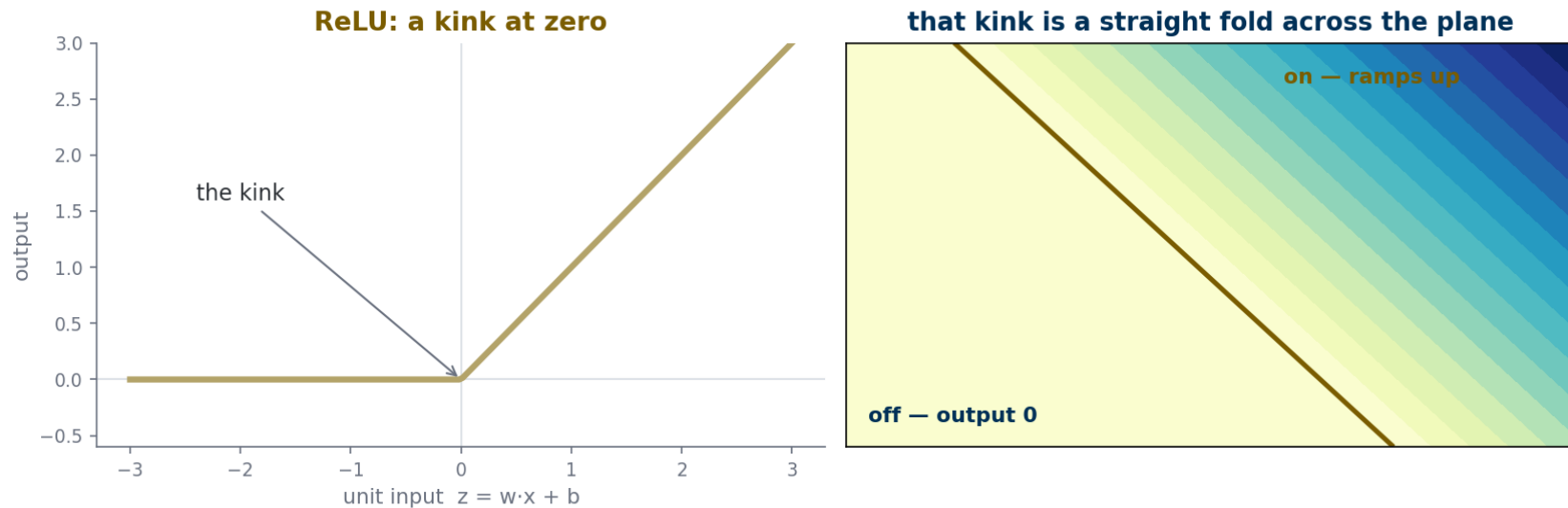


iteration 300 · acc 1.00
wraps the ring



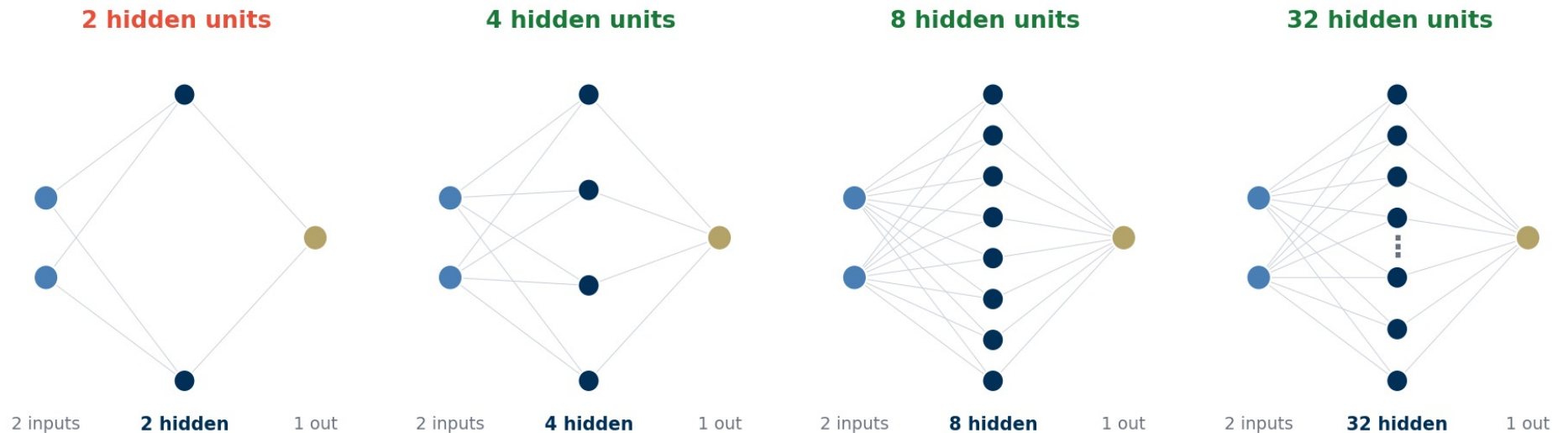
- Just like the logistic-regression line, the network starts knowing nothing — at iteration 3 it calls everything “outer,” so there is no boundary yet and accuracy is a coin-flip 0.50.
- Each gradient step nudges the weights; a small region appears and grows, and its edge bends outward to follow the inner ring — accuracy climbs to 0.72, then 0.94.
- By a few hundred iterations the curve has wrapped the ring completely and accuracy reaches 1.00 — the same “watch it converge” story as logistic regression, only now the boundary is free to curve.

Where the bending comes from: one ReLU, one fold



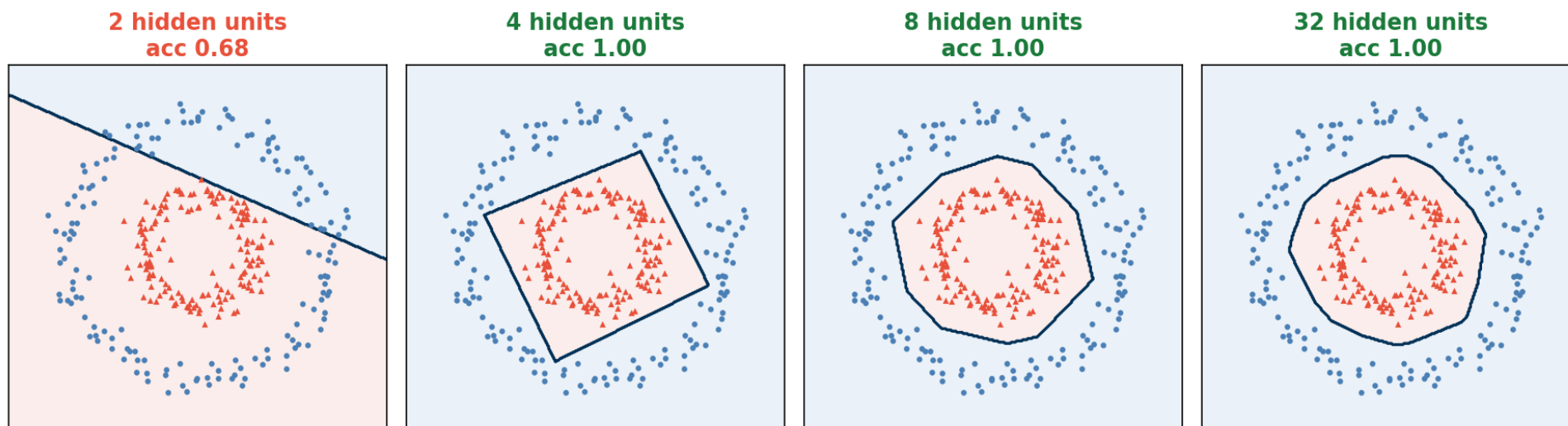
- **Recall the ReLU on the left: it is flat at zero, then bends upward — a single kink.**
- In the input plane, that kink becomes a straight fold line (right): on one side the unit is off and outputs zero; on the other it ramps up. One hidden unit contributes exactly one straight crease.
- **So a hidden layer is a set of these folds — and the output layer decides which side of each fold counts as “inner.”**

What a “hidden unit” is — four networks



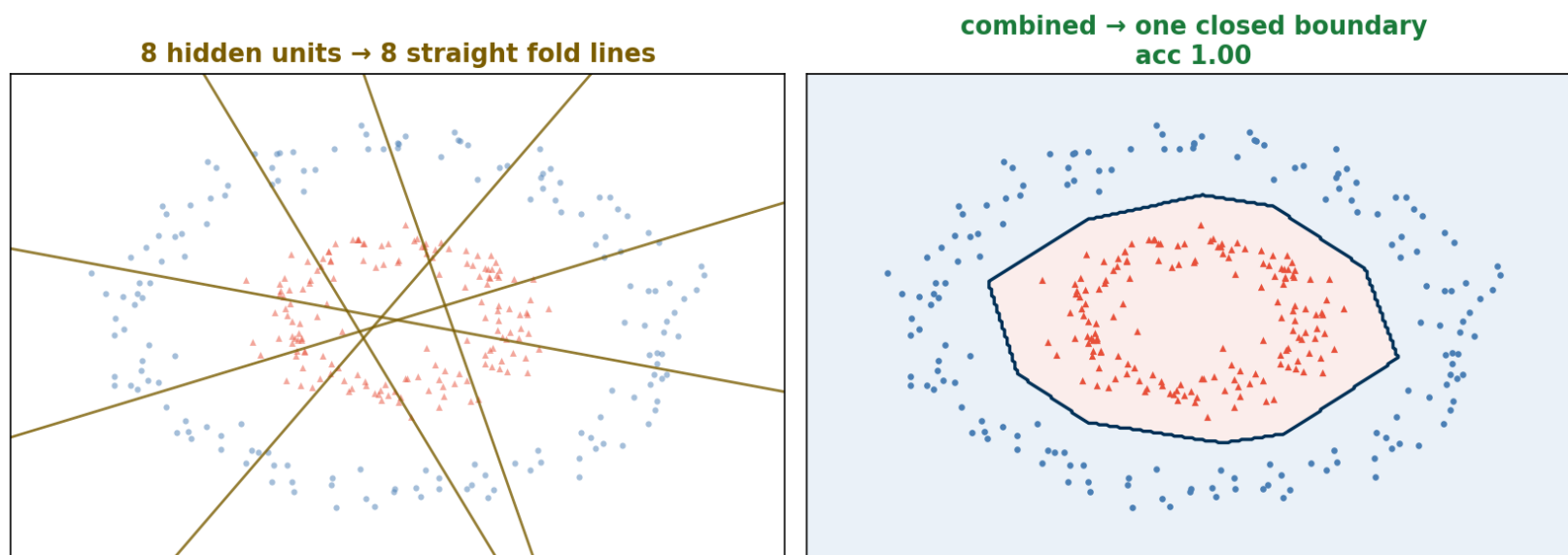
- A hidden unit is one circle in the middle column: it takes a weighted sum of the inputs and passes it through ReLU — the same little unit we drew two slides ago. “Hidden” just means it sits between input and output; we never read it directly.
- How many to use is a design choice. Here are four networks — the same two inputs and one output, but with 2, 4, 8, and 32 units in the single hidden layer.
- Each hidden unit contributes one fold, so this count is exactly how many folds the boundary can have. Next we run each of these four and see the boundary it draws.

More hidden units, more folds, a rounder boundary



- With just 2 hidden units there are only two folds — the best it can do is a straight line, and it cannot enclose the ring at all (accuracy 0.68).
- 4 units give four folds — a quadrilateral that finally encloses the inner ring. 8 units round it into an octagon; 32 units make it look like a smooth circle.
- More hidden units is more folds, and more folds is a boundary that can bend into any shape. That is what a wide hidden layer buys.

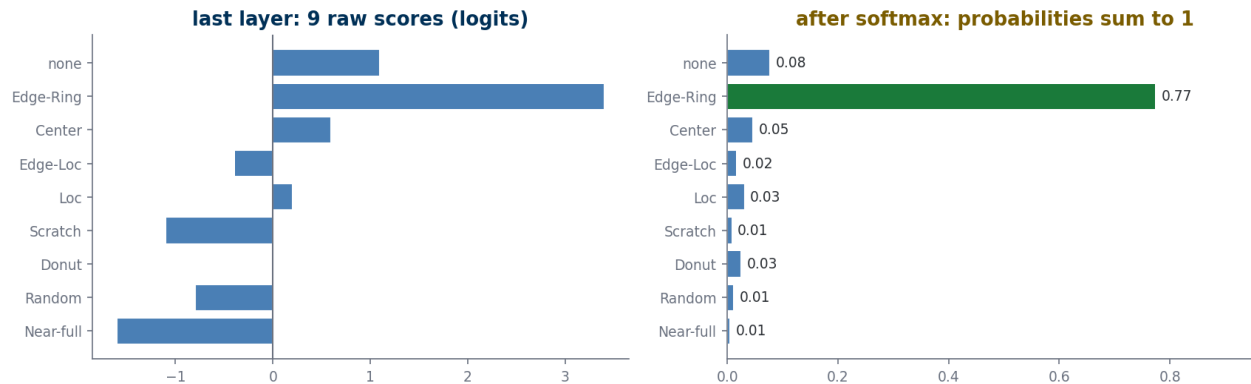
Many folds make the curve



- On the left, the eight straight fold lines from the eight hidden units. On the right, the boundary they produce together: one closed loop around the inner ring.
- Each piece of the boundary is straight; stacked and combined, they approximate a curve as closely as you like — this is why a network can fit almost any shape.

Takeaway A curved boundary is really many straight folds, learned together. Depth and width both add folds — and neither adds any sense of where a pattern sits.

The output head: softmax over nine classes

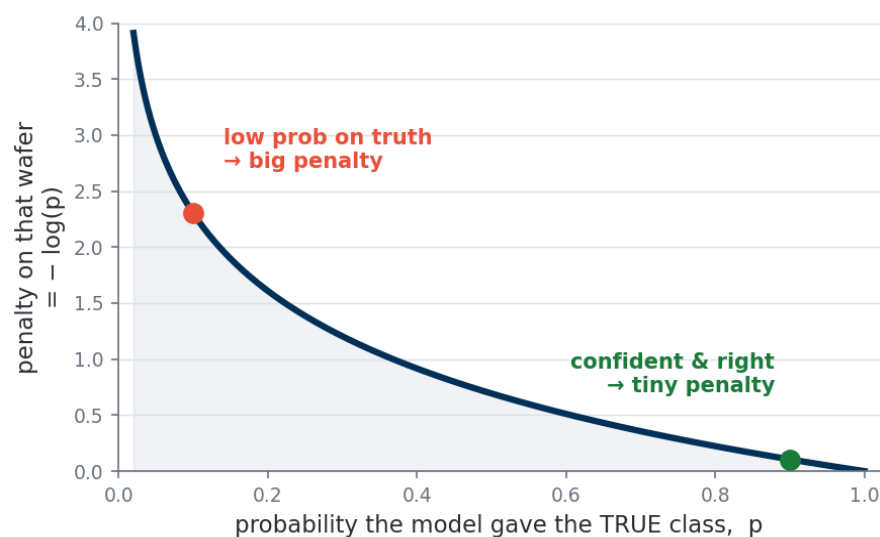


- The last layer emits nine raw scores — one per wafer class. They are unbounded and do not add up to anything.
- Softmax exponentiates each score and divides by their sum, turning the nine numbers into probabilities that are all positive and sum to exactly one.
- The predicted class is the largest probability; here Edge-Ring at 0.77.
- It is the same idea as the sigmoid, generalized from two classes to nine — the multiclass sigmoid.

$$p(\text{class } c) = e^{\text{score}_c} / \sum e^{\text{score}_k}$$

Takeaway Next: how we score those nine probabilities against the truth.

The loss with nine outputs: cross-entropy



two classes (logistic regression):

$$J = - [y \cdot \log(h) + (1-y) \cdot \log(1-h)]$$

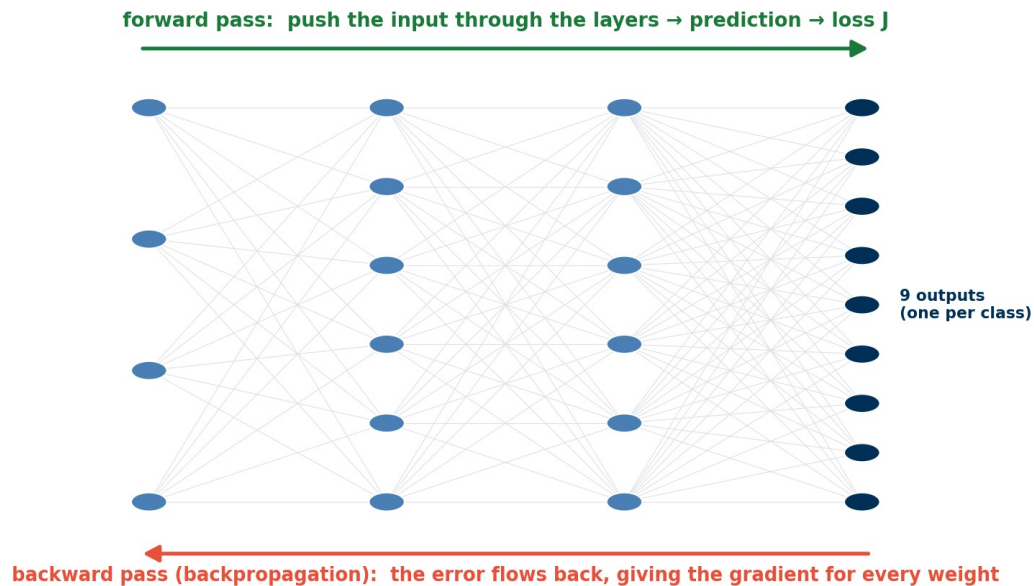
nine classes (softmax):

$$J = - \sum(\text{wafers}) \sum(c = 1..9) y_c \cdot \log(p_c)$$

- The label is one-hot: a 1 on the correct class, 0 on the other eight. Only that one term survives, so a wafer's loss is simply $-\log(p \text{ of its true class })$.
- Confident and correct $\rightarrow p \text{ near } 1 \rightarrow \text{penalty near } 0$. A small probability on the truth $\rightarrow$ a large penalty (the curve on the left).
- It is the exact cross-entropy from logistic regression, summed over nine classes instead of two.

Takeaway One number per wafer — minus the log of the probability it put on the right class — added over the whole training set. Backprop then drives it down.

How a deep network learns: backpropagation



- The loop is the same as logistic regression: predict, measure the loss J , take a gradient step, repeat.
- **Forward pass:** push the input through the layers to get a prediction, then its loss.
- **Backward pass — backpropagation:** starting from the loss, apply the chain rule layer by layer, from output back to input, to find each weight's share of the error.
- With that gradient, every weight takes one downhill step — the same update rule as before, now for millions of weights at once.

Takeaway Backprop is the chain rule run in reverse — it makes gradient descent practical for a network with many layers.

Backpropagation, step by step

- 1 · Error at the output. With softmax and cross-entropy this is remarkably clean — it is just (prediction – truth), the same $p - y$ as logistic regression.
- 2 · Push it back one layer. A hidden layer's error is the error above it, sent back through that layer's weights, times the slope of its activation — for ReLU, 1 where the input was positive and 0 otherwise.
- 3 · Each weight's gradient is the error on its output side times the activation on its input side.
- 4 · Update every weight: $w \leftarrow w - \alpha \cdot \text{gradient}$. A single backward sweep produces every gradient in the network.

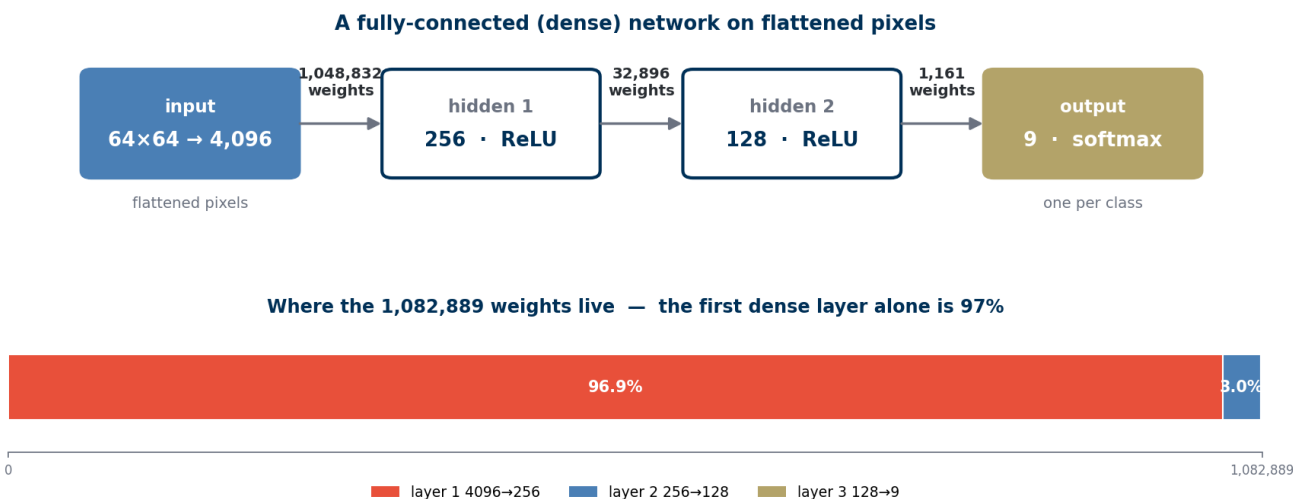
The same four quantities, as math

```
output error:  err = p - y
layer error:   err = (W^T · err_next) * ReLU'
weight grad:   dJ/dW = err · a_in
update:        W ← W - α · dJ/dW
```

* = elementwise · a_in = input activation · ReLU' = 1 if input > 0, else 0

Takeaway “Backward” because each layer reuses the error from the layer above it — that reuse is what lets one sweep compute all the gradients.

The network we actually trained

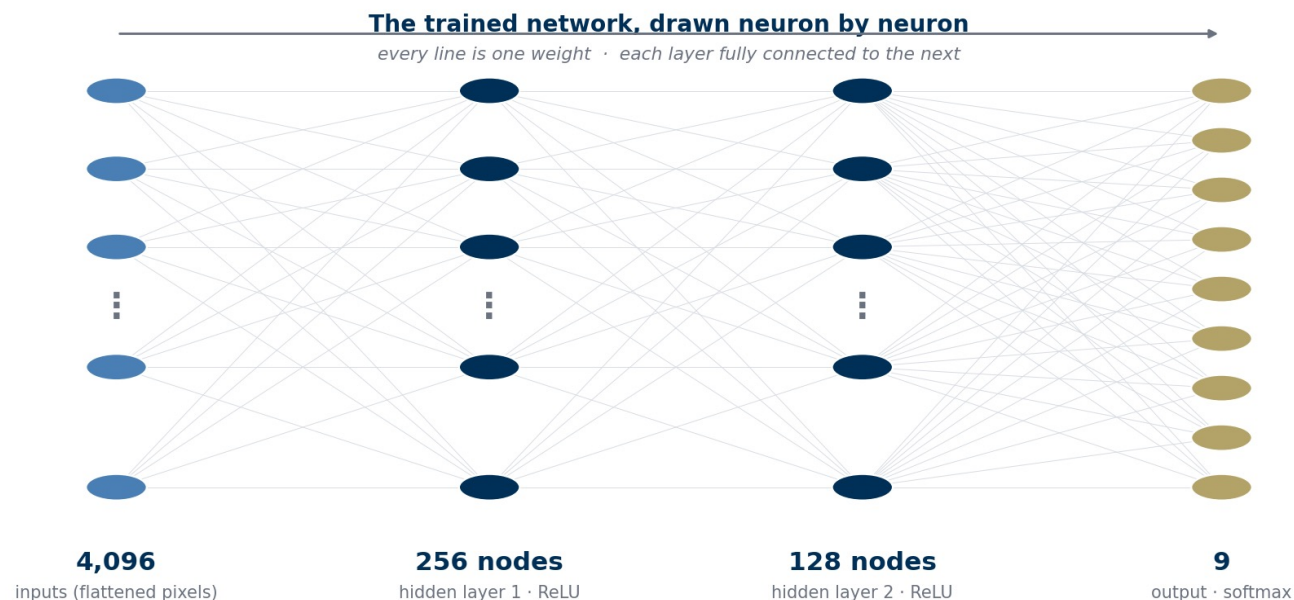


The recipe

- Flatten each 64×64 map to 4,096 inputs.
- Two hidden layers — 256 then 128 — each followed by ReLU.
- A 9-way softmax output; trained with cross-entropy and Adam (a gradient-descent variant).
- 1,082,889 weights in total — and 97% of them sit in the very first dense layer, which reads the raw pixels.

Takeaway A million-weight fully-connected network — far heavier than logistic regression's 37 thousand — and almost all of it is spent connecting every pixel to every hidden unit.

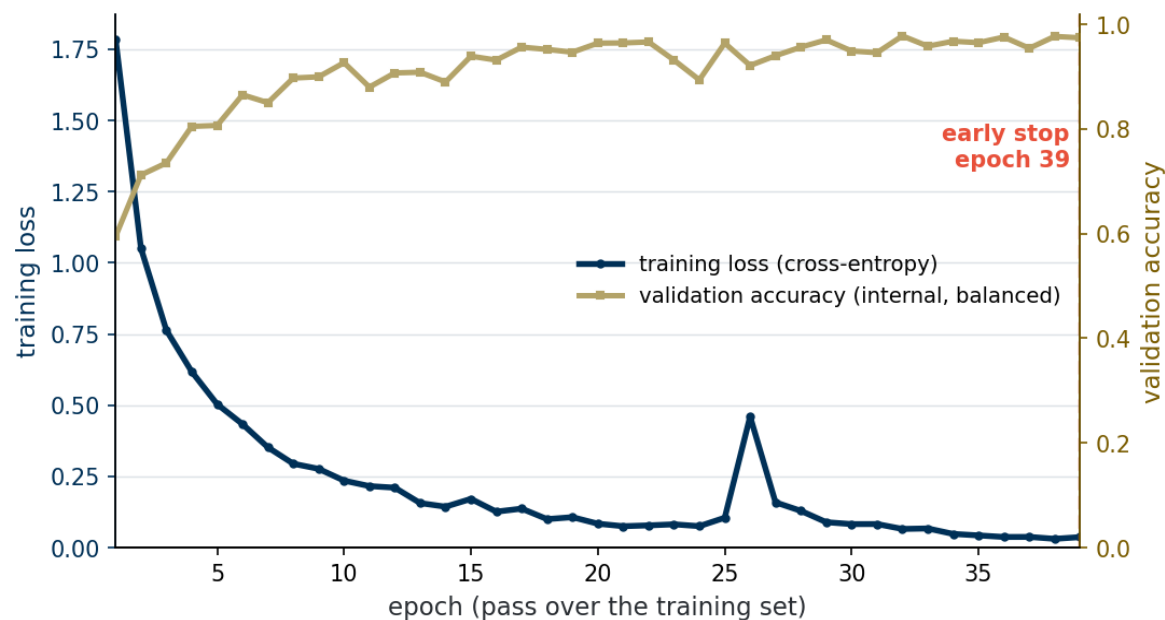
The same network, neuron by neuron



- Four thousand and ninety-six inputs on the left — one per pixel.
- Hidden layer 1 has 256 nodes, each a little logistic unit with its own weights and a ReLU.
- Hidden layer 2 has 128 nodes, wired to all 256 before it.
- **Nine output nodes — one per class — into the softmax.**
- **Every line is a single weight. Fully connecting 4,096 pixels to 256 nodes is what makes that first layer so heavy.**

Takeaway Only a few nodes per layer are drawn; the dots stand for the hundreds more. It is one dense sheet of weights after another.

How training converged

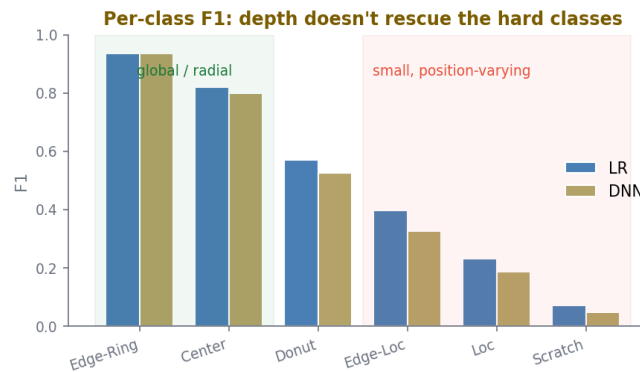
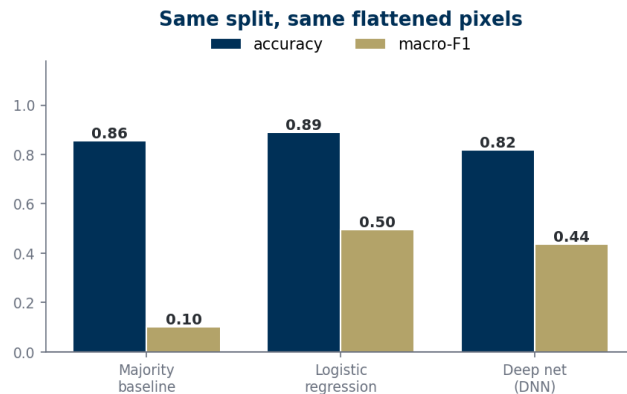


- Each epoch is one full pass over the balanced training set, updating all weights by backprop.
- The training loss falls steeply — from about 1.8 to near 0.04 — then flattens. That flattening is convergence.
- The single spike near epoch 26 is a normal Adam wobble; training recovers within a few epochs.
- **Early stopping halts at epoch 39, when the held-out validation score stops improving, so we do not overfit.**
- **One caution: that validation accuracy is on a balanced internal split. On the real, imbalanced test lots the model gets 0.82 accuracy and 0.44 macro-F1.**

Takeaway The optimization worked — the loss converged cleanly. The ceiling is the model, not the training.

ERA 3 · THE EXPERIMENT

So how does a DNN do on the wafers?



- We trained a real deep network — two hidden layers, 1.08 million weights — on the same lot-grouped split of the development set, flattened pixels, class-balanced.
- **Accuracy 0.82, macro-F1 0.44 — about the same as logistic regression (0.89 / 0.50), and a million-plus parameters bought no real gain.**
- The per-class story is the tell: both nail the global, radial patterns (Edge-Ring), and both collapse on the small, position-varying ones (Loc, Scratch).
- **Depth added flexibility, yet the failures did not move — because the missing ingredient is not**

Takeaway A far bigger, nonlinear model on flattened pixels lands right where logistic regression did. The problem was never flexibility — it is that neither model knows which pixels are neighbors.

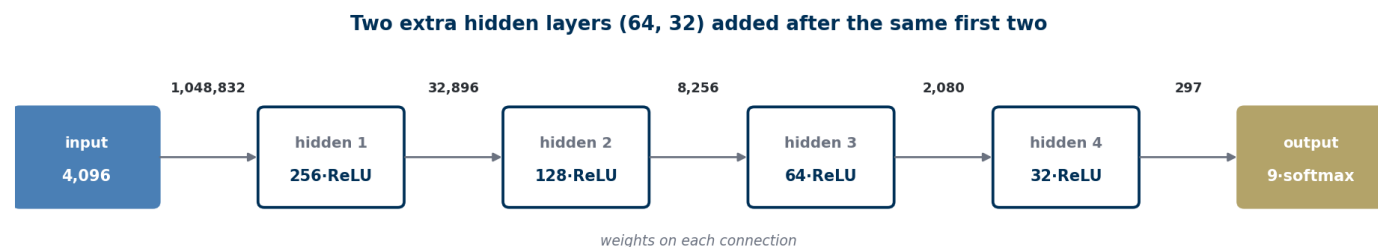
Where the DNN wins and loses, class by class

Class	Precision	Recall	F1	n
none	0.963	0.842	0.898	5,283
Edge-Ring	0.946	0.927	0.936	301
Edge-Loc	0.220	0.625	0.326	184
Center	0.786	0.810	0.798	168
Loc	0.124	0.359	0.185	142
Scratch	0.042	0.057	0.048	53
Donut	0.476	0.588	0.526	17
Random	0.029	0.091	0.043	11
Near-full	0.118	0.333	0.174	6

macro-F1 = 0.437 · accuracy = 0.818 · development split, flattened pixels

- The same winners and losers as logistic regression: Edge-Ring near the top, Loc and Scratch near the floor.
- Scratch precision 0.042: about 23 false alarms for every real scratch it flags — and recall 0.057 misses nineteen of every twenty.
- Global, radial signatures survive flattening; small, position-varying blobs do not. Over a million weights did not change that geometry.
- Macro-F1 0.437 — essentially the logistic-regression baseline of 0.50.

Would more depth help? A 4-layer network



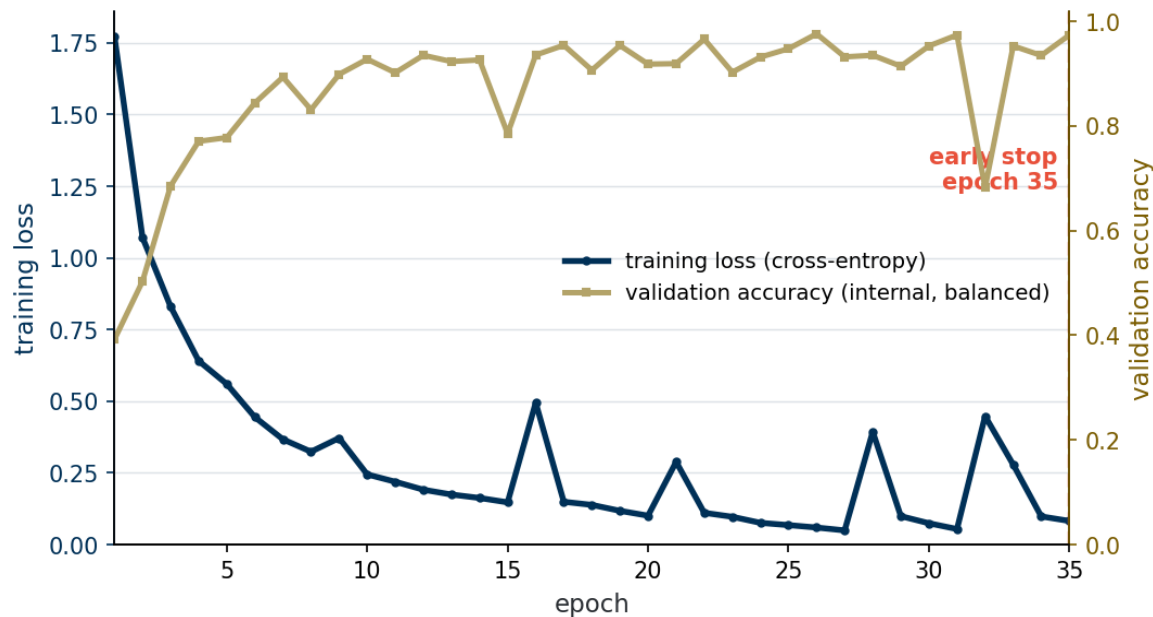
Twice as deep, under 1% more weights — the first pixel-facing layer still dominates



Takeaway “Twice as deep” sounds like a much bigger model, but almost all the weight lives in that first pixel-facing layer either way.

- We keep everything the same and add two more hidden layers — 64 then 32 — after the first two.
- Same flattened pixels, same split, same balanced training and softmax output.
- The parameter count barely moves: 1,082,889 → 1,092,361, under 1% more.
- Why so little? 96% of the weights are still in the first layer that reads the 4,096 pixels; the new 64- and 32-unit layers are tiny by comparison.

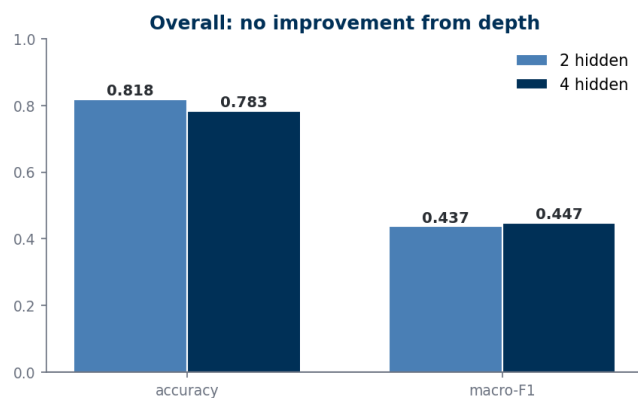
The deeper network trains just fine



- Same training loop, now over four hidden layers of weights.
- The loss falls cleanly — from about 1.8 to near 0.04 — and flattens, exactly like the two-layer run.
- Early stopping halts at epoch 35, a little sooner than before.
- **So optimization is not the problem: the deeper network is perfectly trainable. Whatever limits it, it is not the fitting.**

Takeaway The training curve looks great. Which makes the next slide the interesting one.

Deeper, but no better



- Accuracy actually slips a little, $0.818 \rightarrow 0.783$; macro-F1 barely moves, $0.437 \rightarrow 0.447$ — inside the noise.
- The per-class bars tell the real story: Edge-Ring stays high, and Loc and Scratch stay on the floor.
- The two networks fail on exactly the same classes — the small, position-varying defects.
- Doubling the depth moved nothing that matters.

Takeaway More layers gave the network more ways to bend its boundary — but bending the boundary was never what these classes needed.

Why depth didn't move the needle

Capacity barely changed

The first layer that reads the 4,096 pixels already holds ~96% of the weights. Stacking small 64- and 32-unit layers adds flexibility, not size.

Depth is the wrong medicine

More layers buy a more curved decision boundary. But Loc and Scratch do not fail for want of a curve — they fail because the same defect lands on different pixels.

Same failures, either way

Two layers or four, the model breaks on exactly the position-varying classes. Capacity was never the bottleneck.

The real bottleneck: the model has no idea which pixels are neighbors. Neither width nor depth fixes that — only a layer built for space does. That is the convolution, next.

Powerful, but still blind to space

- A deep network can bend the boundary into almost any shape — far more flexible than logistic regression.
- But to feed it a wafer map, we still flatten the 64×64 grid into 4,096 independent inputs.
- It still has no idea that a pixel sits next to its neighbors, so a defect that shifts position is a brand-new input all over again.
- And a fully-connected layer on 4,096 pixels needs millions of weights — costly to train and easy to overfit.

flatten $64 \times 64 \rightarrow 4,096$ inputs
no sense of which pixels are neighbors

What is still missing: a wiring that looks locally and reuses the same weights across the image, so a pattern is recognized wherever it appears.

Takeaway Depth adds flexibility, not spatial understanding. For images we need a wiring that respects space — the convolution.

PART 1 · SECTION 4

The convolutional network

Give the model the one thing logistic regression lacked: spatial structure.

A short history of convolutional networks



Takeaway The wafer network in this lecture is a small, modern descendant of this line — the same convolution-and-pooling recipe, three blocks deep.

From one neuron to a network

- Stack neurons in layers, with a nonlinearity between them — ReLU: keep positive values, zero the rest.
- **Now the model can bend boundaries and represent complex patterns — a multilayer perceptron.**
- **But on a 64×64 image, a fully-connected layer needs a weight for every pixel-to-neuron link — millions of weights.**
- And it still treats pixels as an unordered list, with no notion of space. We need a smarter wiring pattern.

$$\text{layer} = \text{ReLU}(Wx + b), \quad \text{ReLU}(z) = \max(0, z)$$

The problem with images: fully-connected layers ignore that a pixel's neighbors carry the meaning — and cost far too many parameters.

Takeaway Nonlinearity buys expressiveness; but for images we also need a wiring that respects space. That is the convolution.

A filter is a pattern detector

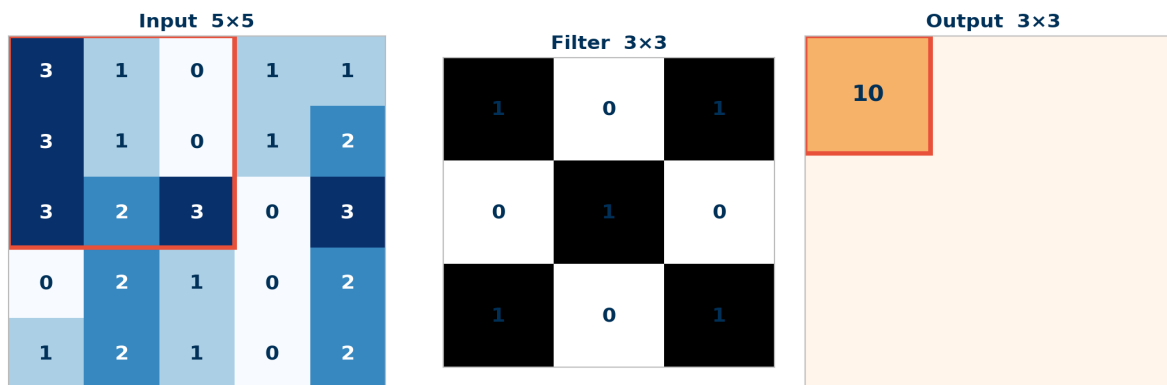


The filter lights up exactly where the image changes — it detects a vertical edge.

- A filter is a small grid of numbers, slid across the image.
- Where the local patch matches the filter's pattern, the output is large; elsewhere it is near zero.
- This 3×3 filter fires on a vertical edge — the output map marks exactly where the image changes.
- Different numbers in the filter → a different detector (horizontal edge, blob, corner).

Takeaway The output is a “feature map”: a picture of where this one pattern occurs across the whole image.

The convolution operation, step by step



Line up the filter on a patch → multiply matching cells → add them up → one output number. Then slide over.

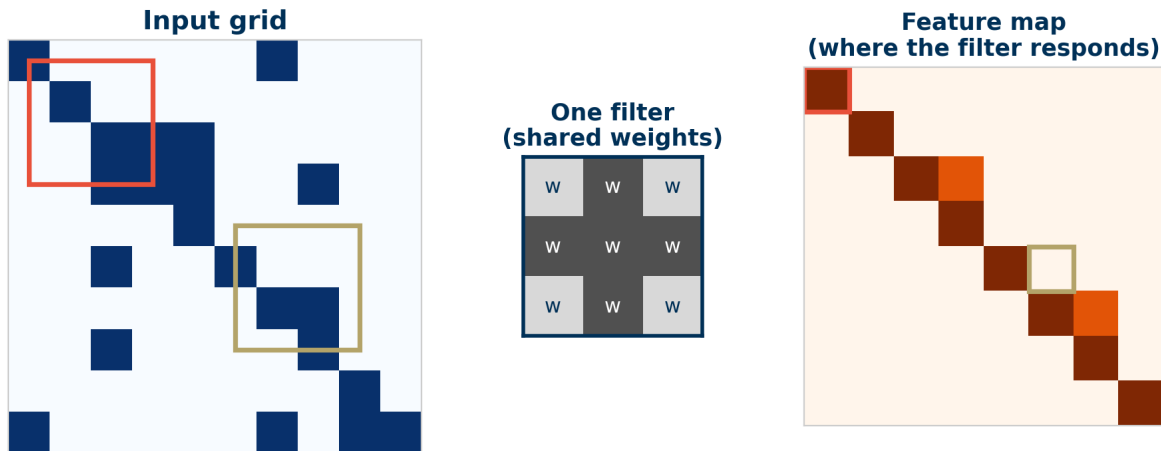
- Line up the filter on a patch; multiply matching cells; add them up → one output number.
- Slide one step over and repeat, covering the whole grid.
- With an $n \times n$ input and an $f \times f$ filter, the output is $(n-f+1)$ wide.

$$(I * K)(i, j) = \sum I(i+a, j+b) \cdot K(a, b)$$

output size = $n - f + 1 = 5 - 3 + 1 = 3$

Takeaway That is the whole operation — repeated at every position.

The filter's numbers are learned weights



- We do not hand-pick the filter. A 3×3 filter is just 9 numbers — weights the network learns.
- They are trained by gradient descent — the same loop as logistic regression.
- Trained on data, early filters end up detecting edges and spots; nobody programmed that.
- **Logistic regression learned one weight per pixel; a conv layer learns a few small filters and reuses them everywhere.**

The same small filter slides across every position — one detector, reused everywhere.

Takeaway A convolution is a perceptron with shared weights, applied at every location — and the weights are learned.

Padding: keep the size, use the border

Input 5×5
→ conv 3×3 → output 3×3 (shrinks)

2	3	3	3	2
1	2	3	3	3
3	2	2	1	1
1	2	2	2	3
3	3	3	3	1

Zero-padded 7×7
→ conv 3×3 → output 5×5 ("same")

0	0	0	0	0	0	0
0	2	3	3	3	2	0
0	1	2	3	3	3	0
0	3	2	2	1	1	0
0	1	2	2	2	3	0
0	3	3	3	3	1	0
0	0	0	0	0	0	0

- Each convolution shrinks the map by (f-1) and touches border pixels only lightly.
- **Fix both by padding the border with zeros before convolving.**
- "Valid" = no padding (the map shrinks). "Same" = pad so the output size equals the input size.

$$\text{output} = n + 2p - f + 1$$

$$\text{"same": } p = (f - 1) / 2$$

Stride: convolution that also downsamples

Input 7×7, filter 3×3, stride 2

3	3	0	1	2	2	2
0	1	0	3	0	1	3
2	2	1	3	2	2	1
1	2	2	3	2	1	3
0	1	1	3	1	2	0
3	1	1	0	2	2	1
3	2	0	0	1	0	2

Output 3×3
steps of 2 → smaller map

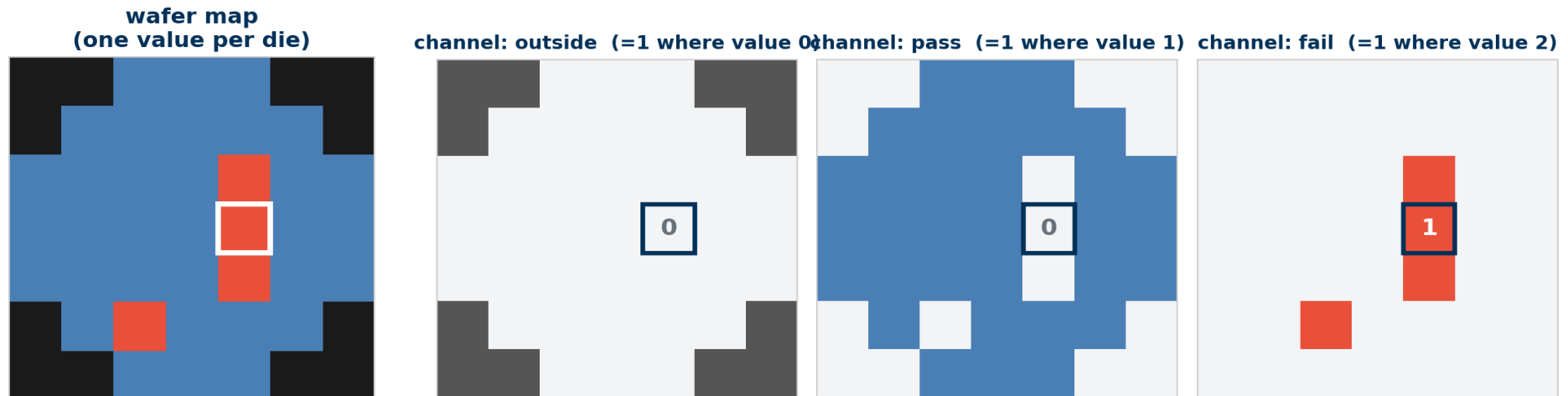
12	12	15
12	18	14
12	9	11

Stride = how far the filter jumps each step. Bigger stride → smaller output.

- Stride = how many cells the filter jumps each step.
- Stride 2 roughly halves the map in each direction.
- It is a cheap way to shrink the representation without a separate pooling layer.

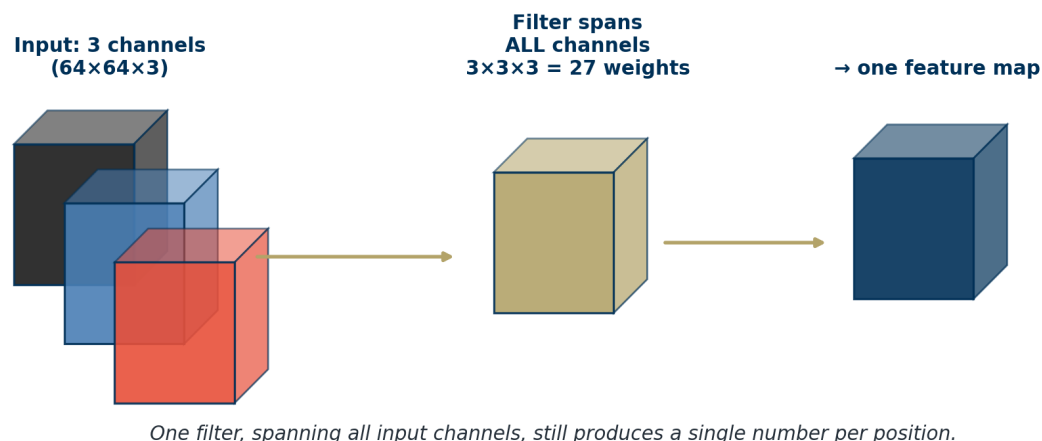
$$\text{output} = \lfloor (n + 2p - f) / s \rfloor + 1$$

Why the wafer map has three channels



- Each die holds one value — outside, pass, or fail. We could label them 0, 1, 2 (or -1, 0, 1); they are three categories, not points on a scale.
- **A convolution multiplies and adds its inputs, so it would read "2" as twice "1" — treating a fail as a big pass. These are kinds, not amounts, so that is wrong.**
- **So we split the single value into three yes/no layers — channels — one per category. A fail die becomes 0, 0, 1: off, off, on.**
- The network input is therefore $3 \times 64 \times 64$, and each filter reads all three channels at once.

Convolutions over volumes (channels)



One filter, spanning all input channels, still produces a single number per position.

- Real inputs have channels. Our wafer map has 3: outside / pass / fail.
- A filter spans all input channels — a 3×3 filter over 3 channels is 27 weights — and still outputs one number per position → one feature map.
- Use K filters → K output channels: a new “depth” for the next layer to work on.

```
channels: torch.stack([x==0, x==1, x==2], dim=1) # outside / pass / fail
```

One convolutional layer — and how few parameters

- A layer applies K filters (each spanning the input depth), adds a bias to each, then ReLU.
- 32 filters $\rightarrow$ 32 output channels.
- The parameter count is tiny — and independent of the image size, because the filter is reused everywhere.

```
layer = ReLU( conv(x) + b )
```

Layer	Weights
Conv: 32 filters of $3 \times 3 \times 3$	896
Fully-connected: $64 \times 64 \times 3 \rightarrow 1000$	≈ 12.3 M

Takeaway Parameter sharing is why a CNN has ~hundreds of weights per layer where a dense net would need millions.

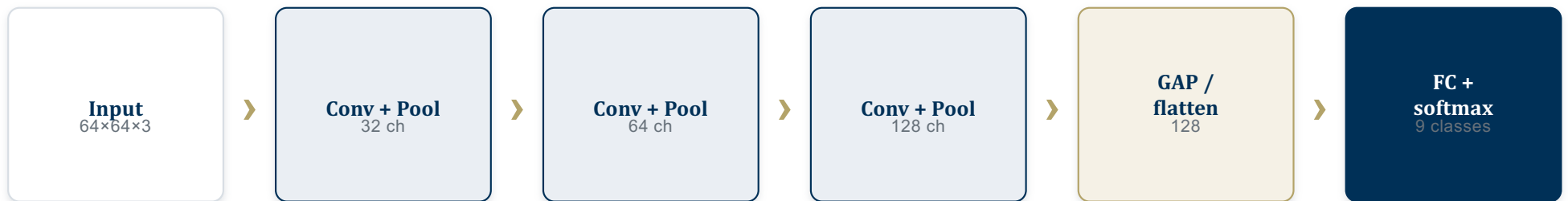
Pooling layers

Input 4×4				Max-pool 2×2 (take the largest in each block)	
0	4	6	5	4	8
4	2	8	2	8	6
7	3	3	6		
8	7	5	0		

- Pooling summarizes small windows to shrink the feature maps.
- Max pooling keeps the strongest response in each 2×2 block; average pooling takes the mean.
- It has no parameters to learn — a fixed operation.
- It also adds a little tolerance to small shifts — more translation invariance.

Takeaway Pool = “was this feature present nearby?”, discarding its exact position. That tolerance is exactly what Loc and Scratch needed.

A complete ConvNet



- Stack [conv → ReLU → pool] blocks, then flatten (or global-average-pool) and finish with a fully-connected softmax.
- **Spatial size shrinks while channel depth grows — the network trades “where” for “what.”**
- **Feature hierarchy: edges & spots → arcs & segments → whole patterns (rings, blobs, lines).**

Takeaway This is the LeNet → AlexNet template. Our wafer model on the next slide is exactly this shape.

Why convolutions win — and how they learn

Parameter sharing

A detector useful in one spot is useful everywhere, so the same filter is reused across the image — giving translation invariance and far fewer weights.

Sparse connections

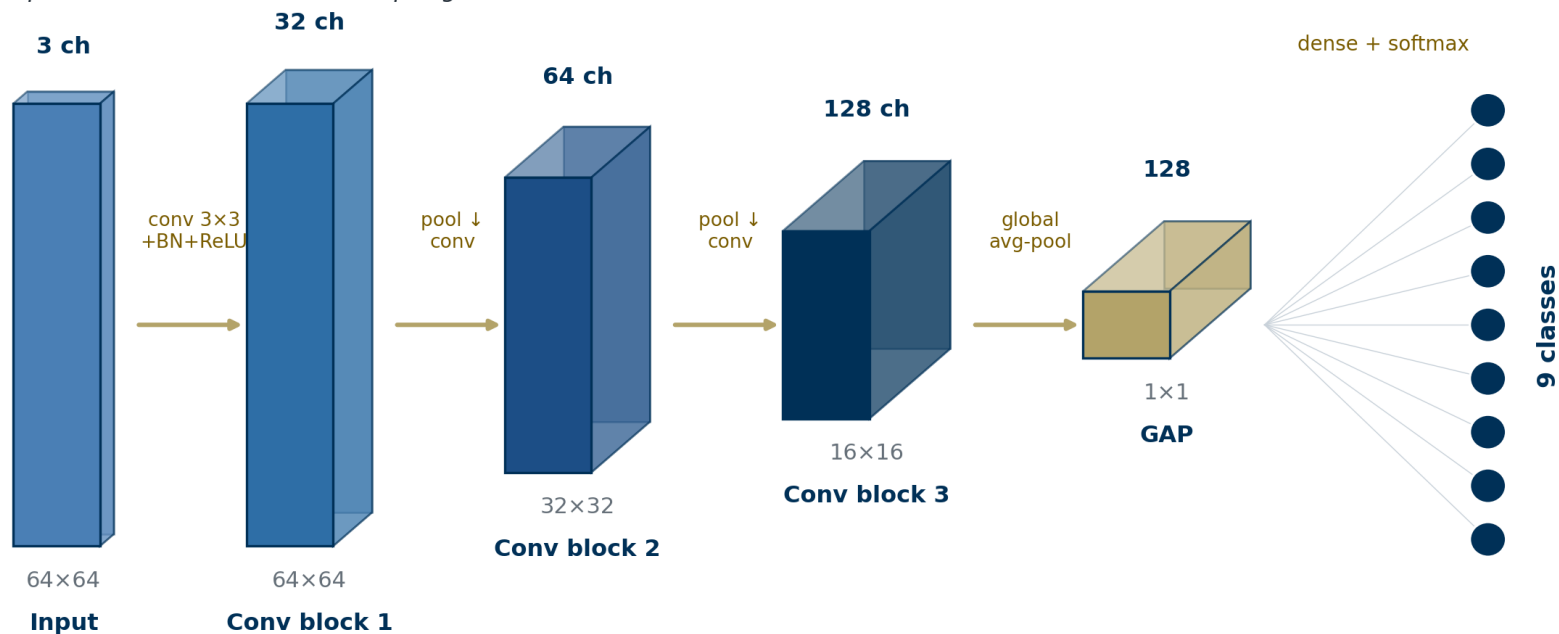
Each output depends only on a small local patch of the input, not the whole flattened image — building in locality.

- **These two are the built-in assumptions (inductive bias) that logistic regression lacked.**
- Training uses the same recipe as logistic regression — a loss plus gradient descent. Backpropagation just carries the gradient back through the layers to every filter.

Takeaway From “one weight per pixel” to “a few shared filters, learned end-to-end” — same training loop, a far better model for images.

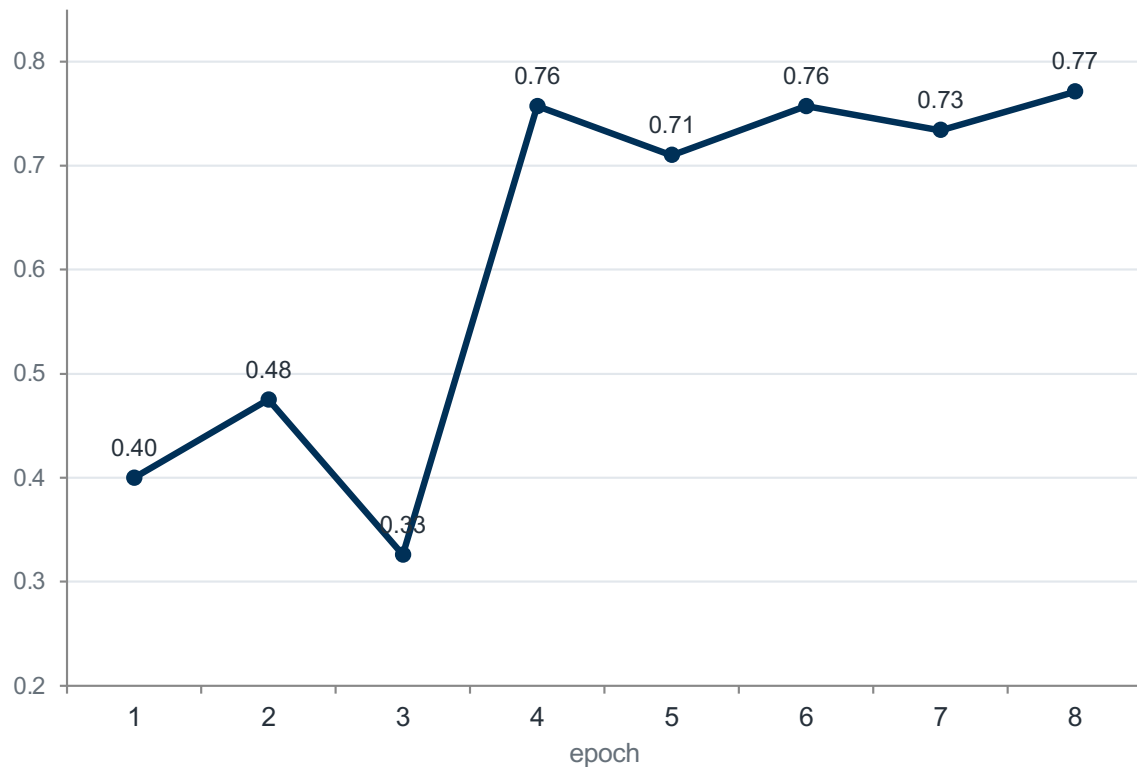
The architecture, as a network

Spatial size shrinks, channel depth grows — the network trades “where” for “what.”



Takeaway Each block = two 3x3 convolutions + BatchNorm + ReLU, then downsample. GAP replaces a giant flatten — keeping translation invariance and slashing parameters. AdamW + OneCycle, 8 epochs.

A non-monotonic curve is not a broken run



- Dev macro F1 climbs, then drops hard at epoch 3 (0.76 $\rightarrow$ 0.33).
- That dip is OneCycle hitting its peak learning rate — expected, not a bug.
- It recovers and finishes at 0.771, the best epoch.
- Watch the trajectory, not a single epoch. Stop only when the trend plateaus.

CNN results: the story seems to end here

0.939

accuracy

0.777

macro F1

Class	Prec	Recall	F1
none	0.995	0.946	0.970
Edge-Ring	0.975	0.975	0.975
Edge-Loc	0.551	0.807	0.655
Center	0.766	0.941	0.845
Loc	0.480	0.724	0.578
Scratch	0.405	0.890	0.557
Near-full	0.800	1.000	0.889

- Every class is now detected. Macro F1 jumps 0.473 → 0.777.
- Scratch recall reaches 0.890 — the linear model managed 0.300.
- Precision on ambiguous classes (Edge-Loc, Loc) is still the soft spot.
- Students think the problem is solved. Part 2 disagrees.

The 14× slide

Scratch precision 0.029 → 0.405 across one architecture change

14 ×

jump in Scratch precision

0.300 → 0.890

Scratch recall also rose

no

accuracy / recall tradeoff

The linear model was not limited by data or by imbalance. **It was limited by having no notion of spatial structure.** Give the model locality and translation invariance, and the class it could not see becomes one it catches nine times out of ten.

WHERE WE STAND

Scoreboard, end of Part 1

Model	Parameters	Accuracy	Macro F1	Scratch F1
Majority baseline (all none)	0	0.854	~0.10	0.000
Logistic regression	36,873	0.688	0.473	0.053
DNN — 2 layers †	1,082,889	0.818	0.437	0.048
DNN — 4 layers †	1,092,361	0.783	0.447	0.060
CNN	≈ 94,400	0.939	0.777	0.557

Status Best macro F1 so far: 0.777 — and the CNN reaches it with ~94K weights, an order of magnitude fewer than the DNN's ~1.1M. Structure, not capacity. It looks like we are done.

Next lecture: 28 hand-built features. 18 seconds of compute. Accuracy 0.952 — higher than your CNN.

† DNN rows use the 26K-wafer development subset; LR and CNN are full-set, so DNN accuracy is not comparable across the line — on a matched split the DNN only ties LR. CNN params: the 32→64→128 + global-average-pool net.

END OF PART 1

Physics beats parameters, until it doesn't

Part 2 opens with the reversal — and then reveals what 28 features cannot do.